XLOG

A Universal GPU-Native Engine for Neurosymbolic Integration

BrainyBlaze team
Technical Whitepaper

July 2026

Abstract

We present xlog, a universal GPU-native engine for neurosymbolic integration: it couples neural perception with the full spectrum of symbolic reasoning—deterministic Datalog, probabilistic inference, and epistemic reasoning over world views—on a single CUDA runtime, keeping all semantic state device-resident with zero host–device data-plane transfers on production paths. Where prior neurosymbolic systems bridge a neural network to a single symbolic backend and keep that backend on the CPU, xlog integrates neural networks, as ordinary predicates, with every reasoning mode through one uniform, transfer-free mechanism. Two device-resident mechanisms make the integration sound and fast. First, a fully GPU-native knowledge-compilation pipeline (provenance → CNF → Decision-DNNF → arithmetic circuit) turns a network's outputs into a differentiable circuit whose forward pass is exact weighted model counting and whose backward pass yields exact gradients on the exact-inference path (a GPU Monte Carlo path covers programs too large to compile); every compiled circuit is proven, by an on-GPU CDCL solver, logically equivalent to its source formula, making the circuit's equivalence to its source a machine-checked certificate rather than an assumption. Second, gradients cross the neural–symbolic boundary zero-copy through DLPack into PyTorch's autograd. We report single-system ablations on one development GPU: circuit caching yields a 2.74× end-to-end training speedup (95% CI $[2.29, 3.18]$) on MNIST addition, and a worst-case-optimal join subsystem yields a 27.96× geometric-mean speedup over xlog's own binary-join baseline on skewed program-analysis workloads. Controlled head-to-head comparisons refine this picture honestly: matched-protocol MNIST addition trains 2.8× faster per epoch than Scallop at equal accuracy, exact inference is correctness-equivalent to but slower than ProbLog2 on small programs, and fused worst-case-optimal triangle counting runs 12–42× faster than Soufflé on skewed graphs while holding peak memory to megabytes where materialization exhausts the device. A public video-event benchmark exercises the same path end to end: on a direct per-timestep target the rule search returns the same induced two-clause theory whether its proximity predicate is hand-set geometry or a network trained through the logic's credit alone—never shown a distance label—at no cost in held-out accuracy. The central contribution is the unified, transfer-free neurosymbolic architecture.

1 Introduction

Neurosymbolic systems combine neural perception with symbolic reasoning, but the two halves run on different machines. Deep learning frameworks—PyTorch, JAX—execute dense tensor computation on the GPU through optimized CUDA kernels. Logic engines—Datalog evaluators, probabilistic-logic systems such as ProbLog [1], answer-set solvers—are CPU-bound interpreters and compilers. Every system that couples the two, from DeepProbLog [2] to NeurASP [3], therefore straddles the PCIe bus: each training iteration ships a network's outputs to a CPU logic engine, materializes query results or proofs, and pulls gradients back to update parameters. At scale—millions of ground atoms, thousands of steps—these host–device transfers, not the inference or learning itself, dominate wall-clock time and bandwidth.

The transfer wall is one half of the problem; partial coverage is the other. Existing neurosymbolic frameworks bridge a neural network to a *single* symbolic backend—DeepProbLog to probabilistic logic, NeurASP to answer-set programming—and run that backend on the CPU. GPU logic efforts, conversely, accelerate one paradigm in isolation: deterministic Datalog [4, 5] or SAT [6], with no neural interface. No system makes the *whole* symbolic spectrum—deterministic, probabilistic, and epistemic reasoning—GPU-resident while coupling it to neural networks with exact, end-to-end gradients through knowledge compilation.

xlog closes both gaps. It is a GPU-native logic programming language whose compiler and runtime treat a neural network as an ordinary predicate: the network's softmax output becomes a distribution over weighted facts that flow into whichever reasoning mode a program uses, and gradients flow back, all without leaving the device. Three mechanisms make this integration broad and sound. A *device-resident symbolic substrate* keeps fact stores, deltas, circuits, solver state, and gradient buffers in GPU memory throughout evaluation, so the symbolic half never crosses the bus. *Verified knowledge compilation*

turns a network's outputs into a differentiable arithmetic circuit and proves, on-device, that the compiled circuit is logically equivalent to its source formula—so the compilation back-end is machine-checked rather than assumed correct. *Zero-copy differentiable coupling* exports the resulting gradients through DLPack [7] directly into PyTorch's autograd graph. The system is implemented in Rust with custom CUDA kernels organized into a layered crate architecture; host involvement is limited to orchestration, I/O, and compilation.

The principal contributions of this paper are:

- **A unified neurosymbolic integration model.** A neural network is a predicate, and the same uniform, transfer-free interface feeds its outputs into deterministic, probabilistic, and epistemic reasoning, with end-to-end gradient training realized through the probabilistic (knowledge-compilation) path—rather than a bespoke bridge to one backend (Sections 2 and 6).
- **A GPU-resident symbolic substrate.** Seminaive Datalog evaluation with stratified negation and aggregation runs as custom CUDA relational kernels, extended with a worst-case-optimal join (WCOJ) subsystem that avoids the intermediate-relation blowup of binary-join plans on skewed cyclic queries (a measured 27.96× geometric-mean ablation, Section 4).
- **GPU-native verified knowledge compilation.** A provenance→CNF→D4→circuit pipeline runs entirely on device, and an on-GPU CDCL solver proves each compiled circuit logically equivalent to its source formula (the structural invariants for correct weighted model counting hold by construction). To our knowledge this is the first GPU-resident knowledge-compilation pipeline with on-device equivalence verification, and it is the paper's central technical result (Section 5).
- **End-to-end differentiable training with zero-copy interop.** Compiled circuits are cached across iterations and evaluated as level-parallel forward/backward kernels, with gradients exported zero-copy via DLPack and Arrow; circuit caching yields a measured 2.74× training speedup. Term embeddings and differentiable ILP ride the same device-resident path (Section 6).
- **Perception learned through symbolic credit on an external benchmark.** On the CAVIAR video-event corpus, under a direct per-timestep target, the rule search returns the same theory whether its proximity predicate is the hand-set geometric one that dedicated rule learners are handed or a network trained by the logic's own credit alone—never shown a distance label or any target of its own—at no cost in held-out accuracy; separately, Event-Calculus rules over the precomputed predicate are induced under fully pre-registered gates, and substituting the learned detector inside *that* search is where the result currently fails, which we report rather than omit (Section 7).
- **Reasoning beyond probability.** Epistemic reasoning over world views (`know`/`possible` under founded and Gelfond-style semantics) and well-founded semantics execute on the same substrate over finite supported programs, including positive modal fixpoints and supported recursion through negation evaluated by GPU-backed well-founded semantics (Section 11)—reusing the CDCL, join, and circuit machinery rather than a separate engine (Section 8).

When xlog pays off. The transfer wall xlog removes bites hardest where the symbolic workload is large and repeated: program-analysis queries over millions of ground atoms (points-to, call-graph, reachability), probabilistic logic with many training iterations that share one compiled circuit, and knowledge-graph reasoning with trainable entity embeddings. In these regimes the per-iteration host–device round-trips of a CPU-symbolic / GPU-neural pipeline dominate wall-clock time, and keeping the whole pipeline device-resident is decisive. Small illustrative tasks such as MNIST addition exercise the integration end-to-end but sit below this regime: they demonstrate correctness and the circuit-caching effect, with the transfer advantage present but secondary—a measured ≈10% of a training step at a standard batch, rising with the per-step neural→symbolic fan-out (Section 9.6)—rather than the dominant factor it becomes at scale. xlog is, accordingly, for practitioners on NVIDIA GPUs whose working set fits in device memory (Section 11) and who need neural perception coupled to *exact, verifiable* symbolic inference rather than fuzzy relaxations.

The remainder of this paper is organized as follows. Section 2 presents the integration model and system architecture. Section 3 describes the xlog language. Section 4 details the GPU-resident symbolic substrate. Section 5 presents verified knowledge compilation, the central contribution. Section 6 ties the pieces together as end-to-end neurosymbolic learning. Section 7 puts that surface on an external benchmark, inducing Event-Calculus rules while the perceptual predicate beneath them is learned through the logic credit alone. Section 8 extends the substrate to epistemic and other reasoning modes. Section 9 reports evaluation results, Section 10 surveys related work, and Section 11 discusses limitations and future work.

2 Integration Model and Architecture

2.1 The Integration Model

In xlog a neural network *is* a predicate. A declaration binds a network to a predicate symbol; evaluating that predicate runs a forward pass, and the network's softmax over a label set becomes a distribution over weighted facts. Those facts enter whichever reasoning mode the program uses—deterministic joins, probabilistic knowledge compilation, or epistemic world-view search—through the same relational interface, and the gradient of a query's value with respect to the network outputs flows back across the same boundary. The integration is *uniform* (one mechanism, independent of backend) and *transfer-free* (the facts, the compiled circuit, and the gradients never leave the GPU). The rest of this section describes the substrate that makes both properties hold: a strict GPU-residency contract, a layered crate architecture, a compilation pipeline, and an extensible IR stack.

2.2 GPU Residency Model

xlog enforces a hard guarantee: all runtime semantic state resides in GPU device memory, or the system returns a deterministic error. There is no silent out-of-core spilling and no implicit CPU fallback; a workload that exceeds the memory budget raises `RESOURCE_EXHAUSTED` with diagnostics rather than degrading transparently. The contract extends to production query paths, which target zero device-to-host (D2H) transfers: the kernel provider accounts for transfers at byte granularity through atomic counters, so a performance-critical section can be bracketed and asserted to download nothing—the check the training path relies on to prove its gradient path stays on-device.

The payoff is bandwidth asymmetry. PCIe 4.0 ×16 delivers roughly 32 GB/s, while GPU HBM provides 1–3 TB/s. A single unnecessary D2H round-trip for a moderate relation (10M tuples at 16 bytes, ≈160 MB) costs about 5 ms—enough to dominate a semi-naive iteration that completes in microseconds on-device. Keeping fact stores, deltas, circuit values, solver state, and gradient buffers exclusively on the GPU eliminates this class of bottleneck; host involvement is restricted to orchestration, I/O, and compilation.

2.3 Crate Architecture

The workspace is organized into five tiers with strict layering: no crate depends on a peer or a higher tier (Figure 1). A leaf *core* crate defines the shared scalar types, the kernel-provider trait, error and memory-budget types, and a bidirectional symbol table for dictionary-encoded strings. Above it, a tier of intermediate representations and device services wraps CUDA via cudarc [8], stages kernel modules (cubin-first with portable PTX fallback), and exposes Arrow and DLPack interoperability. The three core subsystems—the typed language frontend (Section 3), the GPU runtime executor, and the GPU CDCL/local-search solver—compose into integrated pipelines for deterministic, probabilistic, and neural-symbolic execution, which a PyO3 [9] extension module (`pyxlog`) and a command-line binary surface to users.

2.4 Compilation Pipeline

A program is compiled to a GPU-executable plan in five stages. **Parsing** (a PEG parser built with pest) produces an AST covering the full surface, including probabilistic facts, annotated disjunctions, epistemic literals, and neural-predicate declarations. **Dependency analysis** builds a graph with positive, negative, aggregate, probabilistic, and modal edges and computes its strongly connected components (SCCs) with Tarjan's algorithm. The analysis establishes a deterministic stratum order for the ordinary monotone core and classifies supported nonmonotone components for their dedicated plans. **Lowering** fans out from the normalized AST into three reasoning IRs: deterministic and reduced ordinary rules become relational-IR trees, probabilistic provenance becomes PIR, and modal programs retain their semantics in EIR. Neural predicates compose with the applicable route. SAT/MaxSAT and verification use a separate shared service: callers supply a device-resident `GpuCnf` to the CDCL solver rather than creating another reasoning representation or lowering to XGCF. **Optimization** applies cost-based predicate pushdown and join ordering, promotes eligible multiway rules to the WCOJ subsystem, and rewrites bound recursive queries via magic sets (Section 4). **Plan assembly** emits the route-specific execution plan, whose relational work is dispatched to CUDA kernels; supported negated recursion uses GPU-backed well-founded semantics rather than the ordinary semi-naive schedule.

2.5 Reasoning IRs and Circuit Format

Three reasoning IRs keep the backends decoupled; probabilistic knowledge compilation also emits a downstream circuit format. Each form has one role.

RIR (Relational IR) is the algebraic tree—`Scan`, `Filter`, `Project`, `Join`, `GroupBy`, `Union`, `Distinct`, `Diff`, `Fixpoint`, and the worst-case-optimal `MultiWayJoin`/`ChainJoin`—carrying cardinality, skew, and delta-vs-full metadata. Deterministic execution and reduced ordinary subprograms consume RIR; the other backends preserve additional semantics in their own IRs.

PIR (Provenance IR) is the weighted Boolean formula (`Lit`, `NegLit`, `And`, `Or`, `Decision`) derived from provenance tracking over a probabilistic program, capturing its structure without execution order.

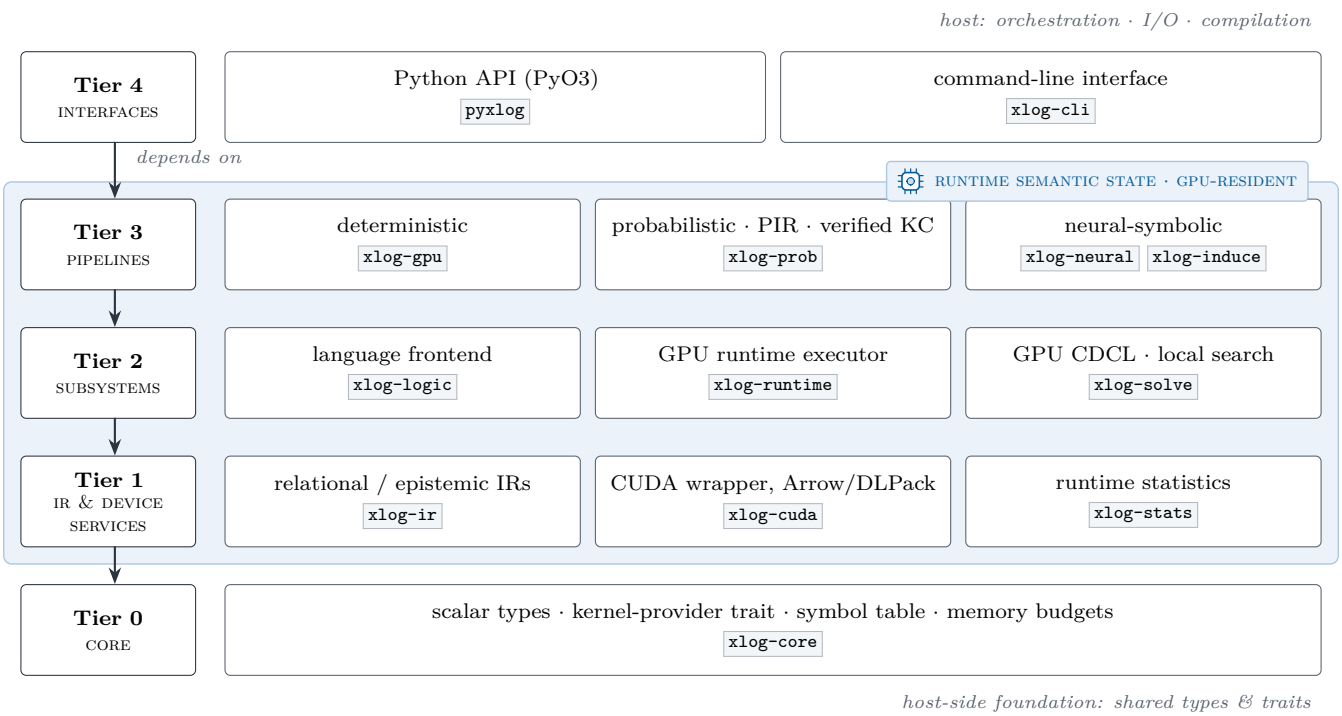

Figure 1: xlog crate hierarchy, with the workspace's production crates in place. Each tier depends only on lower tiers; the shaded device plane marks the tiers whose runtime semantic state is GPU-resident, while the host provides orchestration, I/O, and compilation. Test-only crates (`xlog-cuda-tests`, `xlog-integration`) are omitted.

XGCF (xlog GPU Circuit Format) is the compiled, device-resident form: a levelized DAG whose level offsets let every node at one topological level evaluate in a single kernel launch, with forward (log-space weighted model counting) and backward (adjoint) passes.

EIR (Epistemic IR) preserves modal operators for world-view reasoning and lowers to a dedicated GPU execution plan rather than ordinary relational algebra (Section 8).

GpuCnf and CDCL form a shared solver service, not a fourth reasoning IR. Callers provide a device-resident CNF formula for SAT/MaxSAT search or verification. The CDCL service consumes that formula directly; XGCF remains the circuit format produced only by probabilistic knowledge compilation.

3 The xlog Language

This section describes the xlog surface: types, expressions, user-defined functions, modules, and stratified aggregation. Probabilistic facts (`p::f.`) and neural predicate declarations (`nn/k`) are language-level constructs covered in Sections 5 and 6.

3.1 Design Principles

Three principles shape the surface, each a deliberate enabler of GPU execution rather than a concession. **Typed predicates:** every predicate has a statically known signature over a closed set of scalar types (`u32`, `u64`, `i32`, `i64`, `f32`, `f64`, `bool`, `symbol`), supplied by a `pred` declaration or inferred during compilation, so argument mismatches are caught before any kernel is generated and allocation stays bounded and columnar. **Classified dependencies:** negation, aggregation, modal operators, and recursion interact through SCC analysis on the predicate dependency graph. The ordinary deterministic core requires a monotone or stratified schedule, while supported nonmonotone probabilistic and epistemic components enter dedicated plans, including GPU-backed well-founded semantics for supported recursion through negation. **One language for many paradigms:** probabilistic facts, annotated disjunctions, neural predicates, and SAT constraints share the syntactic core of deterministic Datalog; parsing, normalization, and dependency analysis are shared, while backend-specific lowering preserves the semantics needed by each plan. Unsupported shapes and unbounded terms are rejected before execution rather than entering an implicit fallback.

A minimal program exhibits the core surface:

```
// Typed predicates, facts, a rule, a query.
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
reach(X, Z) :-reach(X, Y), edge(Y, Z).

?-reach(1, N).
```

Listing 1: Minimal xlog program: typed predicates, facts, a recursive rule, a query.

3.2 Type System

Every predicate is assigned a schema over the eight scalar types, with `f32`/`f64` following IEEE 754. A `pred` declaration supplies that schema explicitly; when it is omitted, compilation infers the schema from facts, rule heads, typed body columns, and arithmetic bindings. String literals (`"alice"`) map to `symbol` through a global symbol table, giving the runtime dense integer identifiers while preserving source-level readability; `domain` declarations introduce named aliases. Type consistency is enforced by predicate schemas and lowering-time checks: variable occurrences must agree with known column types, constants are validated against their destination columns, and arithmetic expressions preserve scalar types.

```
pred node(u32, symbol).
pred connected(u32, u32).
pred bridge(u32, u32).

node(1, "alice"). node(2, "bob").
connected(1, 2).

bridge(A, B) :-
  connected(A, B),
  node(A, _), node(B, _).
```

Listing 2: A well-typed xlog program: typed bridge predicate.

Declaring `bridge` so that its head argument is constrained at two incompatible types (e.g. `symbol` in the head but `u32` through `connected`) rejects the program at compile time.

3.3 Expressions, Arithmetic, and User-Defined Functions

Rule bodies may contain arithmetic, comparisons, and calls. Arithmetic bindings use the `is` operator (`D is X + Y`); the right-hand side supports `+ - * / %`, the built-ins `abs`, `min`, `max`, `pow`, `cast`, and user-defined functions. Comparisons compile to `Filter` nodes and push below joins where profitable. Float comparison uses a total ordering—`NaN` and signed zeros yield deterministic results rather than contaminating downstream aggregations.

```
pred point(u32, f64, f64).
pred close(u32, u32).

point(1, 0.0, 0.0).
point(2, 0.5, 0.5).
point(3, 5.0, 5.0).

close(A, B) :-
  point(A, Xa, Ya),
  point(B, Xb, Yb),
  A < B,
  D is (Xa - Xb) * (Xa - Xb)
     + (Ya - Yb) * (Ya - Yb),
  D < 1.0.
```

Listing 3: Arithmetic in rule bodies: pairwise proximity via squared Euclidean distance.

A user-defined function (UDF) is introduced with `func`, with optional parameter types and a return type after `->`. Arithmetic bodies (optionally with `if/then/else`) inline into the same `Expr` trees as built-in arithmetic and participate in the same predicate-pushdown and expression-level optimization. UDFs are pure scalar functions and do not recurse through relations, so a UDF call in a recursive rule does not enlarge the derivable set—closing part of the gap to classical Datalog without breaking monotonicity.

```
pred raw_score(u32, f64).
pred norm_score(u32, f64).

func clamp01(X: f64) ->f64 =
  if X < 0.0 then 0.0
  else if X > 1.0 then 1.0
  else X.

raw_score(1, -0.3).
raw_score(2, 0.7).
raw_score(3, 1.4).

norm_score(Id, N) :-
  raw_score(Id, R), N is clamp01(R).
```

Listing 4: User-defined function: `clamp01` normalizes a raw score into the unit interval.

3.4 Modules and Imports

Programs decompose into `.xlog` modules on a search path. The `use` directive loads a module (`use graph.`) or selectively imports names (`use utils/math::{abs, clamp}.`); a `private` modifier hides declarations from importers. The resolver traverses the import graph, detects cycles, validates participating declarations, and performs bounded cross-module schema inference for undeclared predicate contributions before merging them. The resulting logical program then undergoes full compiler type inference, dependency analysis, and route-specific execution planning, so imported rules participate as if written inline.

```
// library: graph_lib.xlog
pred edge(u32, u32).
pred reach(u32, u32).

edge(1, 2). edge(2, 3). edge(3, 4).

reach(X, Y) :-edge(X, Y).
reach(X, Z) :-reach(X, Y), edge(Y, Z).
```

Listing 5: Module `graph_lib.xlog`: predicate declarations and recursive reach rules.

```
// entry: graph_main.xlog
use graph_lib.

?-reach(1, N).
```

Listing 6: Entry `graph_main.xlog`: imports `graph_lib` and queries `reach`.

3.5 Aggregations and Constraints

Aggregation operators—`count`, `sum`, `min`, `max`, `logsumexp`—appear at rule heads with an explicit aggregation variable and lower to `GroupBy` nodes executed as radix-sort-and-reduce kernels; the stratifier rejects aggregation inside a recursive SCC. Integrity constraints are headless rules (`:- body.`) asserting that `body` is unsatisfiable once evaluation reaches fixpoint; a violation raises a diagnostic. `logsumexp` is included because it is the workhorse aggregation for probabilistic circuits, where log-space numerical stability is essential (Section 5).

```
pred edge(u32, u32).
pred degree(u32, u32).

edge(1, 2). edge(1, 3).
edge(1, 4). edge(2, 3).

degree(X, count(Y)) :-edge(X, Y).

:-edge(X, _), not degree(X, _).

?-degree(X, N).
```

Listing 7: Stratified aggregation: per-source out-degree with an integrity constraint.

3.6 Completeness over Finite Domains

The surface extends toward language completeness while preserving GPU-native execution: accepted constructs normalize into the typed AST and lower through their semantic backend, while forms that cannot be lowered safely are rejected at compile time rather than executed on a hidden CPU interpreter. The accepted extensions are finite typed lists (`[]`, `[H|T]`, the `list<T>` column type), statically-resolvable meta-predicates (`ground`, `functor`, `=..`, source-fact `findall`, `maplist`), explicit stratified negation-as-failure over deterministic relations, magic-set rewriting of bound recursive queries, and finite probabilistic aggregates. Deterministic extensions lower to relational joins and aggregations in RIR; probabilistic aggregates preserve their probabilistic intermediate representation. Open-ended generators, dynamic database mutation, unrestricted `call/N`, and runtime-variable predicate names remain outside the contract.

4 GPU-Native Symbolic Substrate

The deterministic Datalog engine is the device-resident core the neural bridge and knowledge compiler build on: it is what keeps the symbolic half of a neurosymbolic program on the GPU. The algorithmic approach—semi-naive fixpoint iteration over stratified programs—is well established; the contribution here is engineering. Every relational operator runs as a custom CUDA kernel, delta relations are maintained on-device, and no host–device transfers occur during evaluation.

4.1 Semi-Naive Evaluation on GPU

The executor processes an execution plan as strata ordered by the dependency analysis of Section 2. Non-recursive SCCs evaluate each rule once, dispatching its RIR tree to the appropriate kernel and merging results per head predicate. Recursive SCCs run semi-naive iteration (Algorithm 1): each rule is re-evaluated through delta-rewritten variants—one per recursive scan occurrence, so a self-join such as `p(X,Y), p(Y,Z)` contributes two variants—and the new tuples are unioned, differenced against the full relation, and deduplicated, all on-device, until no predicate produces new tuples. The profiler records per-operator timing, row counts, and peak memory, feeding the optimizer.

Algorithm 1 GPU semi-naive evaluation of a recursive SCC. All buffers are device-resident.

Require: SCC rules $\mathcal{R}$; relation store S (device-resident)
Ensure: least fixpoint of $\mathcal{R}$ merged into S

for each rule $r \in \mathcal{R}$ **do**
  $\Delta_r \leftarrow \textsc{Eval}(r, S) \setminus S$ ▷ *seed Δ; set difference on device*
repeat
  for each rule $r \in \mathcal{R}$, recursive scan i in r **do**
    $T_{r,i} \leftarrow \textsc{Eval}(r[\Delta \text{ at } i], S)$ ▷ *one Δ-variant per self-join*
  for each head predicate p **do**
    $\Delta_p \leftarrow \textsc{Dedup}\big(\big(\bigcup_{r,i \text{ for } p} T_{r,i}\big) \setminus S_p\big)$
    $S_p \leftarrow S_p \cup \Delta_p$ ▷ *union + sorted dedup on device*
until all $\Delta_p = \emptyset$ **or** iteration cap reached
free all Δ buffers

4.2 Relational Kernels

xlog implements its relational algebra as direct CUDA kernels rather than wrappers over cuBLAS, cuSPARSE, or Thrust. A library operator imposes its own allocation, synchronization, and host-staging behavior, which would defeat the zero-transfer guarantee of Section 2; bespoke kernels keep every intermediate buffer on-device. Table 1 summarizes them.

Table 1: Core relational CUDA kernels.

Kernel	Purpose
`join`	Hash join with linked-list collision chains and FNV-1a composite hashing over raw key bytes, so all scalar types share one path; count → scan → materialize variants for inner and left-outer joins.
`sort`	Stable 4-bit radix sort (8 passes over 32-bit keys): per-pass histogram, prefix sum, scatter.
`filter`	Comparison operators for column-vs-constant and column-vs-column, with stream compaction via prefix sum.
`groupby`	Sorted-input grouped aggregation: boundary detection, then per-group Count/Sum/Min/Max/LogSumExp reduction.
`dedup`	Sort-based deduplication and set difference with type-aware equality, including IEEE 754 float handling.
`set_ops`	Union (concat + sort + dedup) and set difference via sorted-array binary search.
`scan`	Blelloch parallel prefix sum—the building block for filter compaction, dedup, and group-by.
`pack`	Multi-column key packing into row-major byte arrays with FNV-1a hashing.

The filter kernel makes a correctness-critical choice for floating point. Equality uses standard IEEE 754 semantics (NaN $\neq$ NaN); ordered comparisons use a total-ordering transform that maps an `f64` bit pattern to an `i64` whose integer order matches the IEEE 754 `totalOrder` predicate ($-$NaN $<$ $-$Inf $<$ negatives $<$ $-0.0 < +0.0 <$ positives $<$ +Inf $<$ +NaN), so Datalog programs produce deterministic results for all floating-point inputs.

4.3 Adaptive Join Planning

The optimizer performs cost-based transformations using runtime statistics. Each plan node is costed along four dimensions—expected rows, coordination cost, peak GPU memory, and host–device transfers—with transfers penalized heavily so the planner prefers device-resident plans even at higher raw compute cost. Predicate pushdown decomposes filters above joins into left-, right-, and cross-predicates; join ordering uses dynamic programming up to ten relations and a greedy heuristic beyond, with selectivities learned from prior executions. The executor can export a statistics snapshot and merge it back before the next compilation, closing the loop between execution and optimization.

4.4 Worst-Case Optimal Joins

Binary-join plans can materialize intermediates asymptotically larger than the final result—the classic pathology for cyclic queries such as triangle enumeration. This is not a corner case for xlog's target workloads: program-analysis queries (points-to, call-graph and reachability inference over code-dependency graphs) are dominated by cyclic, skewed joins where the intermediate blowup governs cost. xlog addresses this with a worst-case-optimal join (WCOJ) subsystem [10–12]. A pure-Rust hypergraph planner analyzes rule bodies for multiway-join eligibility, derives a deterministic variable ordering from cardinality and selectivity statistics, and emits a `MultiWayJoin`/`ChainJoin` node when promotion provably preserves output semantics; otherwise the binary-join plan is retained as a certified fallback. The GPU kernel executes the join via count → device prefix scan → materialize over flat sorted-column storage, with a four-byte metadata D2H total and no count-vector transfer. Coverage spans the triangle, 4-cycles, K-clique templates (K=5–8), recursive/SCC integration, and helper-relation splitting that exposes inner skew to root-level histograms. A default-on skew classifier decides per query whether to dispatch WCOJ: high-skew inputs take the WCOJ path, while uniform or empty inputs fall back to the binary-join chain. Section 9 reports the measured speedup.

4.5 Runtime Optimization

Interactive and long-running deployments—REPL sessions, iterative solver loops, incremental training—repeatedly evaluate similar queries over relations that change little between calls. A stateless executor rebuilds join indices and recomputes shared subplans each time, paying the dominant cost repeatedly. Three optimizations exploit this between-call stability, each with explicit controls and zero added data-plane transfers: common-subexpression elimination caches safe deterministic subplans within an evaluation; adaptive re-optimization adopts a compiler-supplied candidate plan when misplan telemetry crosses a configurable ratio, with output-equivalence checks and rollback; and a persistent hash-index manager—keyed on relation generation, schema, and device—retains built indices across repeated sessions with deterministic LRU eviction (Section 9).

4.6 Reversible Symbols

Datalog programs operate on strings, but GPU kernels operate on fixed-width numeric columns. xlog interns each unique string to a dense, sequential `u32` identifier and resolves it back in $O(1)$ at output time. Symbol columns then *are* ordinary `u32` columns—join, sort, filter, and

dedup kernels operate on them with no special-casing—and the string table stays host-side, since variable-length data is needed only at ingestion and output. The `symbol` type maps to Arrow's `Dictionary(UInt32, Utf8)`, so export to columnar consumers (cuDF, Polars, DuckDB) is zero-copy while preserving the compact internal representation.

5 Verified Knowledge Compilation

Knowledge compilation is the bridge that makes a neural network and a logic program differentiable end-to-end. A network's softmax becomes a distribution over weighted facts; provenance tracking over the program yields a weighted Boolean formula; and that formula compiles into an arithmetic circuit whose forward pass is exact weighted model counting (the query probability) and whose backward pass yields exact gradients with respect to the weights—hence with respect to the network outputs. Probabilistic systems such as ProbLog [1] follow this compile-once/evaluate-many model on the CPU, so a neural pairing like DeepProbLog [2] ping-pongs circuit values and gradients across the PCIe bus every iteration. xlog runs the entire pipeline on the GPU—and, crucially, *proves the compiled circuit correct on-device*, so the bridge is sound rather than assumed.

5.1 The Compilation Pipeline

The pipeline transforms a probabilistic program into a GPU-resident arithmetic circuit (the XGCF format) through five stages (Figure 2); each is one or more CUDA kernel launches, with all intermediate state device-resident.

Provenance extraction. Provenance over deterministic evaluation produces a weighted Boolean formula [13]. An on-device interner hash-conses node batches through an atomic GPU hash table, deduplicating structurally identical subformulas; this is essential because grounding can produce exponentially many redundant subformulas, and interning on the host would force the uninterned graph into host memory—reintroducing the very transfer the pipeline exists to avoid.

CNF encoding. A Tseitin [14] transformation lowers the formula to CNF on the GPU, using parallel prefix sums to assign compact, gap-free variable IDs and emit clauses into a device-resident compressed-sparse-row structure.

D4 compilation. The CNF compiles into a Decision-DNNF circuit [15] via a GPU-native variant of the D4 algorithm [16]: a BFS frontier exposes parallelism, per-block DFS workers perform component detection, decision-variable selection by VSIDS-like activity [17], and recursive decomposition. A smoothing pass forces every branch of an OR/decision node to mention the same variable set, and the result is levelized so each topological level evaluates in one kernel launch. Decomposability, determinism, and smoothness—the structural properties that make weighted model counting over a Decision-DNNF correct [18, 19]—are thus established by construction of this lowering, not by the equivalence check below.

Verification and evaluation are described next.

5.2 Verification: a Machine-Checked Certificate

Rather than trust the compiler, xlog proves the compiled circuit C equivalent to its source formula φ (Algorithm 2). It solves $\varphi \wedge \neg C$ and $C \wedge \neg\varphi$ on a GPU CDCL solver; both must be unsatisfiable for $C \equiv \varphi$. This is a complete formal proof, not a sampling check: UNSAT results carry a resolution proof checked on-device, and the solver never reads its status back to the host—a wrong answer triggers a device-side trap rather than silently propagating. This certifies that the circuit computes the same Boolean function as φ (the count-correctness invariants are guaranteed separately, by construction, as noted above); the compilation back-end thus becomes a machine-checked stage rather than an assumption, closing the failure mode in which a silently miscompiled circuit would corrupt every downstream probability and gradient. The check is on the back-end stage specifically: provenance extraction and CNF encoding are trusted, not certified. CPU-side certified knowledge compilation exists [20]; the contribution here is performing the check on-device, sharing the GPU substrate. The same device-resident CDCL substrate is reused for SAT/MaxSAT queries and for epistemic candidate validation (Section 8).

Algorithm 2 GPU-native verified knowledge compilation. The circuit is accepted only if proven equivalent to its source formula on-device.

Require: weighted Boolean formula φ (device-resident)
Ensure: verified circuit C (XGCF), or device-side trap

$\varphi \leftarrow \textsc{InternHashCons}(\varphi)$ ▷ *dedup via atomic GPU hash table*
$\Phi \leftarrow \textsc{TseitinCNF}(\varphi)$ ▷ *prefix-sum variable IDs; CSR clauses*
$C \leftarrow \text{D4}(\Phi)$ ▷ *BFS + block DFS; VSIDS; smooth; levelize*
$u_1 \leftarrow \text{CDCL}(\varphi \wedge \neg C)$; $u_2 \leftarrow \text{CDCL}(C \wedge \neg\varphi)$
if $u_1 \neq \textsc{Unsat}$ **or** $u_2 \neq \textsc{Unsat}$ **then**
  device-side trap ▷ *miscompilation never reaches host*
check resolution proofs of u_1, u_2 on-device
return C ▷ *certificate: $C \equiv \varphi$, machine-checked*

The verified circuit supports a level-parallel forward pass (log-space weighted model counting, one kernel per level, yielding $\log Z$) and a reverse-order backward pass that accumulates per-variable gradients. Both leave their results in GPU-resident buffers consumable directly by PyTorch via DLPack (Section 6).

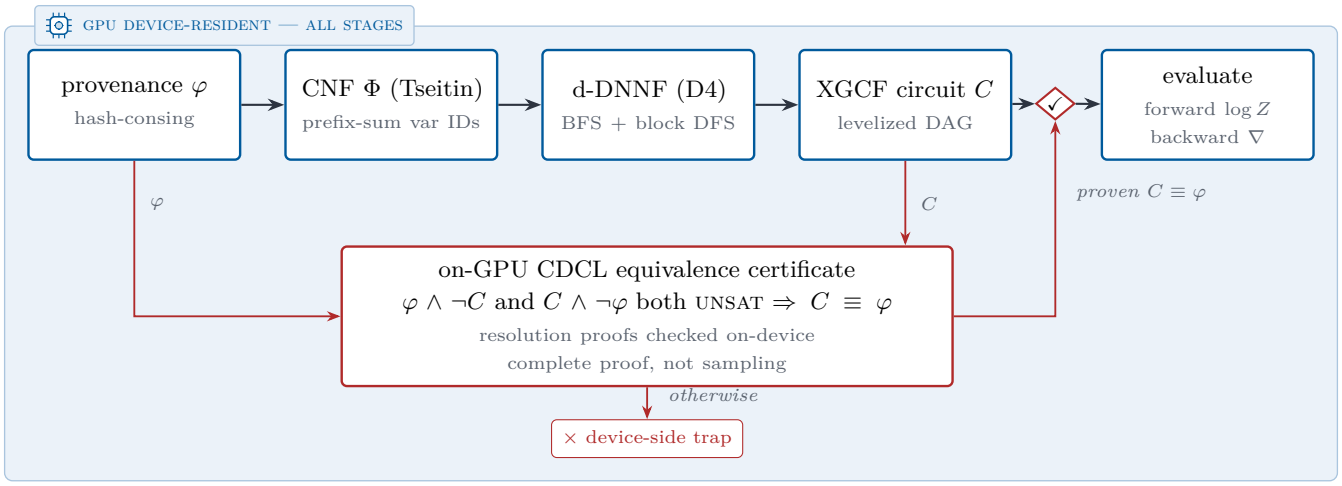

Figure 2: Verified knowledge-compilation pipeline. Every stage runs as CUDA kernels on device-resident state. The compiled circuit C reaches evaluation only through the equivalence gate (✓): an on-GPU CDCL solver proves $\varphi \wedge \neg C$ and $C \wedge \neg\varphi$ both unsatisfiable, and a failed proof halts in a device-side trap instead of reaching evaluation.

5.3 Circuit Caching

D4 compilation is the most expensive stage—on the order of a minute for a cold start on the MNIST-addition benchmark—and a naive implementation pays it every epoch. The key observation is that the circuit *structure* depends on the propositional structure of the grounded program and the evidence pattern, but *not* on the numerical weights. When neural parameters change between iterations, the weights on probabilistic facts change while the Boolean formula and its compiled circuit stay identical. The compiled circuit is therefore cached, keyed by a structural hash of the provenance graph and evidence configuration; subsequent iterations update only the weight buffers and run the forward/backward kernels. On MNIST addition this amortization yields a 2.74× end-to-end training speedup (Section 9).

5.4 Monte Carlo Fallback

When exact compilation is infeasible—the ground formula is too large for D4 within the GPU memory budget, or the user accepts approximate results—xlog provides a GPU-parallel Monte Carlo engine. It draws values for all probabilistic facts on the GPU and evaluates the deterministic program in each sampled world, using rejection sampling or, when every evidence atom is a Bernoulli fact, evidence clamping that forces observed values and eliminates rejection waste. Both methods report confidence intervals. The sampler is a GPU-resident megakernel that evaluates each world in a single device launch with a bounded device-side fixpoint; unsupported fragments are rejected fail-closed with a typed diagnostic rather than silently falling back to the host.

6 Neurosymbolic Integration

This section is where the three pillars meet. The device-resident substrate (Section 4) keeps the symbolic state on the GPU; verified knowledge compilation (Section 5) turns a network's outputs into a sound differentiable circuit; and zero-copy coupling carries gradients back into the neural optimizer without leaving the device. Together they let neural perception and logical reasoning train jointly, end to end, with no host–device transfers in the loop.

6.1 Neural Predicates

In xlog a neural network *is* a predicate. The `nn/4` declaration embeds a network directly into the program:

```
nn(network_name, [input_vars], output_var,
   [output_labels]) ::predicate(args).
```

Listing 8: Neural predicate declaration (`nn/4`).

This is not a foreign-function escape hatch: it states that evaluating `predicate(args)` requires a forward pass through the named network, whose softmax over the label set becomes a distribution over probabilistic facts feeding the knowledge-compilation pipeline of Section 5. On the Python side a PyTorch module is registered against the predicate:

```
program.register_network("mnist_net", net, optimizer)
```

Listing 9: Registering a PyTorch module against a neural predicate.

Registration is typed rather than by name. A caller may state the arity of the network's predicate, and that

claim is checked against every `nn/4` declaration bound to the network in the program and rejected on mismatch: the program's own declaration is the authority. Per-argument catalog sort identifiers may accompany the arity—one per declared argument, refused without an arity or at the wrong length—and an opaque content hash may identify the registered artifact, so a retrained network mints a new identity when it is registered again. The engine interprets neither; both are carried for consumers, which read them back through a queryable metadata view together with each declaration's predicate, predicate arity, input arity, and label set. A downstream rule learner can therefore check a candidate against what the program actually declares instead of against a naming convention—the property Section 6.5 relies on.

The dataflow runs in two directions. **Forward:** when evaluation reaches an `nn/4`-backed predicate, the registered module runs, and its softmax vector is decomposed into weighted probabilistic facts—one per label—fed into the PIR/CNF/D4/XGCF pipeline. Because circuit structure depends on the program, not the weights, the compiled circuit is cached and only the leaf weights are updated on later iterations. **Backward:** after the circuit computes the forward log-probability and backward gradients via level-parallel kernels, the per-leaf gradient buffers are exported as DLPack tensors and injected into PyTorch's autograd graph, and the optimizer updates the network as usual. The GIL is released during GPU work, so CUDA execution does not block the interpreter. A neural predicate is not confined to a query head, moreover: it may also appear inside rule bodies, so that whole rules—not just a single perceptual predicate—become the trainable object (Section 6.4).

6.2 End-to-End Training

The canonical example is MNIST addition (the DeepProbLog benchmark): given two digit images, predict their sum, with supervision only on sums—the network never sees individual digit labels. The entire reasoning structure is two rules:

```
nn(mnist_net, [X], Y, [0,1,2,3,4,5,6,7,8,9])
    ::digit(X, Y).
addition(X, Y, Z) :-
    digit(X, D1), digit(Y, D2), Z is D1 + D2.
```

Listing 10: MNIST addition: a CNN-backed digit predicate and an addition rule.

The `nn/4` declaration connects the CNN to `digit/2`; the addition rule computes every consistent decomposition of a sum through probabilistic marginalization. Each query `addition(i, j, s)` trains by minimizing $-\log P(\texttt{addition}(i, j, s))$ under the compiled circuit. The driver compiles the program, registers the network, and runs the loop, which batches queries sharing a circuit template and runs forward/backward over the cached circuit:

```
program = pyxlog.Program.compile(SRC)
program.register_network("mnist_net", net, optimizer)
program.add_tensor_source("train", train_images)
history = pyxlog.train_model_tensor(
    program, queries,
    epochs=epochs, batch_size=batch_size)
```

Listing 11: Python driver: compile, register, train.

The decisive performance property (Figure 3) is that compilation and verification happen once, on the first forward pass; every later iteration reuses the cached circuit, executing only the weight update and the level-parallel forward/backward kernels. This is the mechanism behind the 2.74× training speedup; the loss reduction across seeds confirms that gradients flow from circuit evaluation through the logic program back to the CNN, and the full timing breakdown is reported in Section 9.

6.3 Zero-Copy Interoperability

A GPU-native engine is only useful if results reach the frameworks researchers already use—without copies that would reintroduce the transfer wall. xlog exposes its device-resident state through four standard interfaces.

DLPack. Each relation column or gradient buffer becomes a managed DLPack [7] tensor whose GPU memory is reference-counted and freed only when every consumer releases it; on import, xlog validates device, dtype, contiguity, and alignment, optionally against an expected schema. In Python the protocol surfaces as a capsule, so `torch.from_dlpack` yields a live CUDA tensor backed by the very memory xlog wrote.

Arrow. For columnar tools (Polars, DuckDB, cuDF), relations export through the Arrow [21] C Device Data Interface; symbol columns export as Arrow dictionary arrays, so a consumer reads symbolic columns as native string dictionaries while xlog keeps the compact integer representation.

Python bindings. The `pyxlog` extension (PyO3 [9]) exposes compilation, evaluation, training, and result extraction; tensor-valued results are returned as DLPack capsules, and long-running GPU operations release the GIL so data-loader threads proceed concurrently.

PyTorch autograd. The circuit behaves as a differentiable function inside PyTorch: the network's grad-enabled forward feeds the circuit, the scalar loss is exported via DLPack and re-imported, and a batched path stacks inputs into one forward pass and runs a single backward—so gradients flow from the logic layer back to `nn.Module` parameters with no custom autograd `Function`.

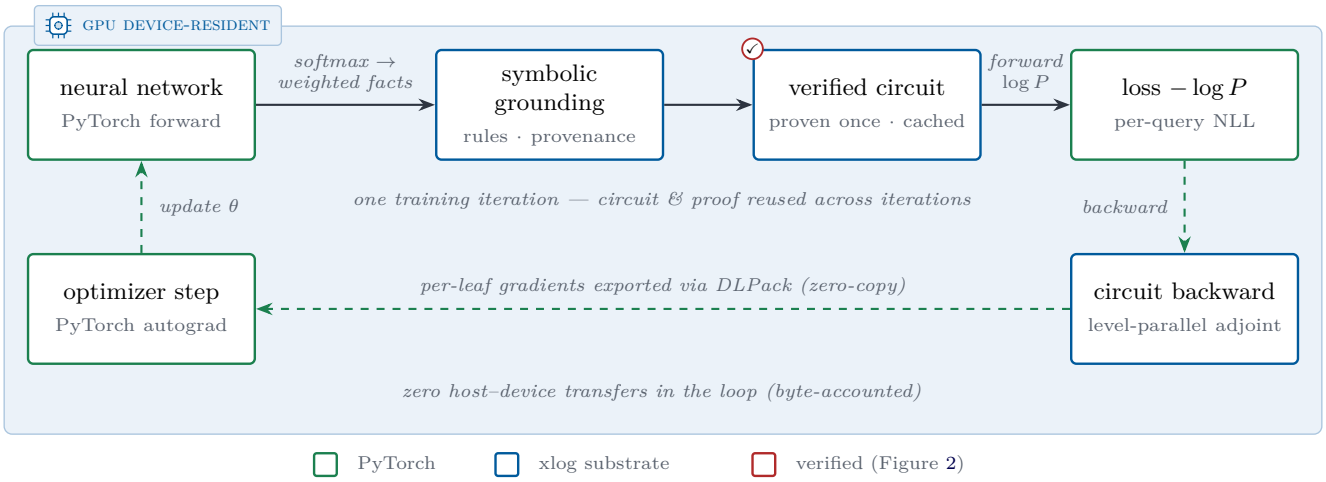

Figure 3: One end-to-end neurosymbolic training iteration. The entire loop—neural forward pass, symbolic grounding, circuit evaluation, loss, and gradient return—executes on one CUDA device (shaded plane) with zero host–device data-plane transfers. The circuit is compiled and proven correct once (✓, Figure 2), then reused across iterations, and gradients re-enter PyTorch's autograd graph zero-copy through DLPack.

6.4 Trainable Rule Bodies

Placing a neural predicate in a rule body lifts xlog from learning a single perceptual predicate to learning whole rules, while the pipeline stays device-resident with no host round-trips.

Mixed bodies. A trainable body may interleave neural-evaluated atoms with ordinary symbolic literals. The symbolic literals act as hard join conditions: they gate which groundings can fire but carry no probability mass or gradient, so the head probability factors as the (satisfiable?) symbolic condition times the neural probability scaled by a learned per-rule weight $\sigma(w)$, and gradients flow only through the network outputs and the rule weight, never through the ground facts.

Existential joins. When an existential (non-head) body variable is a neural predicate's input, xlog grounds that predicate over the *real* join domain *inside* the circuit rather than stripping the join as a pre-filter: the neural occurrence expands into one circuit leaf per domain event, and provenance OR-aggregates $\exists e.\ \text{neural}(e) \wedge \text{join}(e, \mathit{head})$ at each head binding. This makes "does supporting evidence exist?" rules learnable—reasoning over a whole domain of events—rather than only per-instance classification.

Joint multi-rule mixtures. Several same-head trainable clauses combine as a noisy-OR mixture, $P(\mathit{head}_i) = 1 - \prod_k \big(1 - r_k[i]\, m_k[i]\, \sigma(w_k)\big)$, where $r_k[i]$ is a candidate's relational eligibility, $\sigma(w_k)$ its learned rule weight, and $m_k[i]$ an optional graded confidence in $[0, 1]$ carried per binding (for example, a fact's confidence trajectory over time). Rule learning thus moves from binary rule eligibility to a continuous, differentiable weighting of competing and cooperating rules.

Learned body gates. Inside that mixture a candidate may also carry a neural conjunct over a per-binding entity feature $\phi(x)$: its eligibility becomes its relational grounding *and* a straight-through-thresholded gate $g_\theta(\phi(x)) \geq \tau$, so a gated clause competes against ungated ones on the same noisy-OR terms and g_θ trains jointly with the rule weights under the same held-out selector. By default ϕ is detached where it enters the engine: gradient reaches θ and the rule weight but stops at the neural head, leaving the feature producer frozen. Training the producer too is an explicit opt-in per candidate—one flag leaves ϕ attached, so gradient flows on into the backbone that computed it. Only the autograd linkage changes; the values uploaded are identical either way.

These trainable-rule paths share the verified circuit and the zero-copy substrate of the simpler predicate case, and extend to recursive programs now that provenance converges on two-sided recursive strongly-connected components. Section 7 carries the surface onto a public benchmark rather than a demonstration: the perceptual predicate inside an induced Event-Calculus rule body is trained through symbolic credit alone, cross-validated under pre-registered gates. What that section deliberately does not do is rank the surface against a dedicated rule learner—the protocol differences are real and unquantified—and the accuracy and training cost of these paths on standard relational benchmarks remain unestablished by head-to-head comparison. The surface is a beta capability (Section 11).

6.5 Engine-Mode Credit and Candidate Selection

Learning a rule body is not only a gradient problem: it is also a decision about which candidate to keep, and the two need different evidence. Engine-mode training sepa-

rates them. The engine already enumerates the candidate space—the well-formed pairs of body relations over the program's own binary relations—and already holds the join extensions those candidates read, so training scores that space directly rather than a reconstruction of it. A candidate's score on a fact is a noisy-OR over the witnesses the engine reports for that fact: a neural body relation contributes a probability per witness, read at the fact's own label, while a purely relational one contributes a fixed $\{0, 1\}$ cover. Candidates combine as a softmax over their logits, and the resulting negative log-likelihood is a single autograd graph, so one backward pass updates the candidate logits and the network behind the probabilities together. Two disciplines keep the scoring honest. The candidate pool is validated against the typed registry of Section 6.1 rather than trusted by name—a neural relation whose declared arity the credit has no witness semantics for is refused at registration, and a pool that filtering empties reports its per-filter counts instead of silently shrinking. And because raw engine constants index the caller's feature rows directly, that identity is accepted only when the witness domain is exactly the dense row range; a misalignment that would stay in bounds while gathering another event's probability is refused rather than guessed at.

Selection by generalization, not by trained weight. Training credit cannot distinguish a crisp but coincidental rule from a soft but correct one, even in principle; generalization can. Selection therefore never reads the trained weight. The facts are split into folds; each fold trains on the rest and rescores every candidate on the held-out facts by its own witness semantics; the arbiter ranks the fold-averaged scores. A *fit gate* then drops any candidate whose held-out score falls below a threshold—a rule that cannot fit held-out data is not a rule—and ties within the score quantum break toward the relational candidate on Occam grounds, but only when that narrowing leaves exactly one: preferring a relational explanation over a neural one is licensed, choosing among relational duplicates the data cannot separate is not. When neither step is decisive the arbiter returns no rule and says why. Abstention is an outcome of the selector, not a failure of it. The held-out score may be accuracy or F1, and the choice matters: at a rare positive class accuracy sits on the all-negative base-rate plateau, where a candidate that predicts nothing outranks the best real detector, which is why the benchmark protocol of Section 7 selects on held-out F1 instead.

Masked witnesses and withheld evidence. A per-witness mask lets evidence be withheld rather than denied. Any (entity, label) row of the witness space may be marked masked, and a masked witness is removed from the candidate's index when it is built: it contributes exactly zero credit and zero gradient, never a coerced false. Masked is not false, and selection honors the difference. A fact whose score already clears the decision threshold is certain regardless—the noisy-OR is monotone in its witness probabilities, so no completion of the masked ones can pull it back down—as is any fact that had no masked witness at all. Everything else is uncertain: excluded from the candidate's held-out score, because it neither confirms nor refutes, and counted against a coverage fraction. A candidate left with too little certain evidence is dropped by a coverage gate before the fit gate ever runs, and the abstention names it, so no candidate is killed by facts it was never allowed to see. The channel currently rides the accuracy axis; combined with the F1 holdout it is refused up front, since that score derives a fold's measurability from raw labels a mask can hide.

Acceptance without retraining. The same machinery also runs as an acceptance instrument rather than a trainer. Given an externally trained detector, a frozen entry point scores every enumerated candidate against it with no optimizer, no gradient and no folds—nothing trains, so every fact is already held out with respect to this system—and applies exactly the arbiter's gates: coverage, fit, tie semantics, Occam narrowing, abstention. The detector must be in evaluation mode, and a training-mode module is refused rather than quietly switched, since batch-norm statistics mutate and dropout de-determinizes the scores even under a no-gradient scope, both breaking the bit-identical scoring the entry point exists to provide. Where the cross-validated path asks whether a detector can be trained for a rule, this one asks whether a rule is right given the detector one already has.

6.6 Term Embeddings and Differentiable ILP

Two further paths ride the same device-resident bridge. **Term embeddings** attach dense, optionally trainable vectors to logical symbols: where `nn/4` maps perception to discrete facts, embeddings give logical entities a continuous representation that participates in neural computation, with gradients flowing through the embedding lookup—enabling, for example, knowledge-graph entity representations trained jointly with neural perception. **Differentiable ILP** learns first-order rules rather than network weights: candidate rules are represented as sparse bitmasks over the ground-atom space, credit assignment runs entirely on-device via scatter–gather, and a promotion pipeline decides which rules are committed into the program. Six gates always run once training has converged: the discovered rule must derive every training positive and no training negative, pass a novel-fact audit against a bound on unexplained derivations, lose no fact of any protected relation, meet a held-out F1 threshold, and have typed schemas available for the relations it names. Two further gates run when explicit held-out examples are supplied, on those positives and negatives. Convergence is a precondition checked before any gate, not one of them, and the ambiguity scan—which alternative candidates also explain the data—is reported beside the gates but never decides: promotion requires every

gate to pass, and anything else returns for manual review rather than committing.

The permutation-null fit gate that Section 7 relies on is none of these. It belongs to the holdout arbiter of Section 6.5, and it is not an extra gate there either: it supplies that arbiter's fit threshold, deriving it per fold from label permutations instead of fixing it at a constant, so a candidate must beat what shuffled labels alone achieve on its own fold. The two mechanisms judge different objects—one whether a candidate generalizes better than chance, the other whether an already-selected rule may enter the program—and they sit on different paths; the benchmark runs of Section 7 exercise the arbiter, not the promotion pipeline.

Where the trainable rule bodies of Section 6.4 learn rule *weights* and per-binding confidences, differentiable ILP learns rule *structure*; the two are complementary. This sparse, GPU-resident formulation avoids the cubic blowup of dense rule materialization. Unlike the trainable-body surface, this path is not evaluated in Section 7: it remains validated by demonstrations rather than head-to-head benchmarks, and is a beta subsystem (Section 11).

7 Learning Event Rules from Perception

The preceding sections build the machinery; this one puts all of it on an external benchmark at once. The task is symbolic event-rule induction over video surveillance data, and the perceptual predicate the induced rules depend on is itself learned—through the logic credit alone, with no perceptual supervision anywhere in the loop.

7.1 The Problem

The Event Calculus is the standard logical formalism for narrative reasoning over time. A *fluent* is a time-varying property of the world—for instance, that two tracked people are `meeting`. Two event predicates say when a fluent changes: `initiatedAt(F,T)` holds when something at time `T` makes fluent `F` begin, and `terminatedAt(F,T)` when something makes it stop. A domain-independent axiom of inertia closes the loop: a fluent *holds at* a time if it was initiated earlier and not terminated since. Reasoning is therefore cheap and generic once the initiation and termination rules are known; the whole difficulty sits in obtaining them.

Those rules are classically hand-written by a domain expert. The line of systems that learns them instead—inductive logic programming over Event-Calculus theories—learns them from a symbolic input vocabulary that somebody else has already computed. On the CAVIAR video benchmark this means the learner is handed a relation such as `close(P1,P2,T)`, already reduced from raw tracker coordinates by a hand-chosen distance threshold, alongside per-person activity states. Rule induction thus starts one layer above perception, and the quality of the perceptual layer is a fixed input rather than something the learning explains.

7.2 What Is Learned Here

In the runs reported below the proximity predicate is not given. It is `close_nn`, a small network over the raw pair coordinates, and it is never shown a distance label, a threshold, or any target of its own. Its only gradient arrives through the induced rules: a candidate clause containing `close_nn` in its body is scored by how well the resulting Event-Calculus theory explains the frame-level annotations, and that score is what propagates back to the network's weights.

Mechanically this is exactly the trainable rule bodies of Section 6.4, carrying a real benchmark rather than a demonstration. The neural atom sits inside a rule body; the symbolic literals beside it act as hard join conditions that gate which groundings fire without carrying gradient; credit reaches the network through the same verified circuit and the same zero-copy export described there. Structure search and weight training are not two pipelines glued together—the rule learner's own objective is the network's loss.

7.3 Protocol

Three guards define the protocol, and all three are pre-registered—declared before the run they govern, with no configurations executed on the real data outside those declared.

Leak-free scene-family splitting. The distributed CAVIAR corpus used by the published Event-Calculus learners is a dump of 26 video segments, and it is not clean: an independent frame-level audit of every `meeting` transition event in the dump against the original ground-truth XML classifies 21 of 25 events as real, 3 as duplicates (two source videos appear in both the dump's train and its test file) and 1 as a splice artifact; one video alone contributes 855 duplicated gold pair-frames. Cross-validating over segments therefore still permits same-scene transfer. The clean protocol folds over *scene families* recovered from the source XML instead, so no scene appears on both sides of a split.

Recall-aware holdout. Scoring candidates by held-out accuracy is useless at this class balance—roughly ten positive transitions against tens of thousands of rows—because every candidate sits on the all-false base-rate plateau, and the ranking provably inverts: the best real detector scores below the empty theory. Candidate selection uses per-fold held-out F1 instead.

Permutation-null fit gate. A candidate must beat what label shuffling alone achieves on its own fold. Each

fold derives its own threshold from 1000 label permutations (seed 7) as the 95th percentile of the pool-maximum mean per-fold F1, computed per target and on the training side only. A theory whose clauses do not clear that bar is not returned: the search abstains and reports an empty theory rather than committing one. Abstention is a first-class outcome of this protocol, not a failure mode of it.

7.4 Results

Perception learned through logic credit. On the windowed-fold protocol with a direct per-timestep target, the search finds the same two-clause theory on every fold in both the relational and the neural mode:

```
holdsAt_meeting(PP,T) :-both_inactive(PP,T), close*(PP,T).
holdsAt_meeting(PP,T) :-both_active(PP,T), close*(PP,T).
```

where `close*` is the precomputed geometric predicate in the relational mode and the learned `close_nn` in the neural mode. Held-out F1 per fold is 0.9215/0.6804/0.7695 with the precomputed predicate (mean 0.7905) and 0.9215/0.7918/0.7695 with the learned one (mean 0.8276). The learned detector reproduces ground-truth geometry exactly on two folds and exceeds it on the third, where a soft learned boundary survives the train/test geometry shift better than a hard threshold. This is the paper's actual claim on this benchmark—a perceptual predicate trained through nothing but symbolic credit standing in for hand-set geometry without loss—and it rests on no published comparison at all.

Protocol-matched cross-validation. Under the full Event-Calculus target, the combined 26-segment corpus (32,360 co-visible pair-frames, 1,833 gold `meeting` frames) was cross-validated ten-fold by video segment with every gate re-derived inside each fold, and scored by micro-aggregation over pooled per-fold counts, as the published evaluations do. The result is precision 0.6580, recall 0.8271, F1 0.7329 (1516 true positives, 788 false positives, 317 false negatives). The same initiation clause—`both_active & close`—is selected on all ten folds, each time over that fold's own permutation null; the termination theory is empty on all ten, so the fluent persists by inertia to the end of each co-visible pair-run.

Comparison, not ranking. This protocol matches the published Event-Calculus learners on the axis that matters most for the metric: F1 is computed on `holdsAt` inferred through inertia, on the same distributed corpus, with the same micro-aggregation. The published figures for `meeting` on that corpus are 0.735 for hand-crafted rules, 0.782 and 0.792 for OLED [22] (the latter as reported by its authors, the former as re-run in the WOLED paper [23]) and 0.887 for WOLED-ASP; they are tabulated beside ours in Section 9. Our 0.733 sits just under that group's lowest figure. We draw no ordering from any of these differences, in either direction, because five protocol deltas separate the rows and they do not all point the same way. (i) The published runs cross-validate over windowed interpretations with subsampled negatives; ours folds over whole video segments. (ii) They learn full initiation *and* termination programs; the run above has a two-literal initiation clause and an empty termination theory, which costs 788 of 2,304 predicted-positive frames as false positives on the folds with detected interior terminations. (iii) Their rule language admits longer bodies—bottom clauses averaging some fifteen literals against our two-literal cap. (iv) Their settings are reported as the best among several parameter settings tried; ours are pre-registered. (v) Both compute F1 on `holdsAt` through inertia, which is why the comparison is drawn at all. The honest contrast is one of protocols, not of systems.

A second iteration, labeled as such. Adding two pair-level transition relations to the Event-Calculus vocabulary—one for a pair crossing the proximity threshold outward, one for strictly growing distance—and re-running the identical folds, seeds and gates raises the result to precision 0.7357, recall 0.8260, F1 0.7782 (1514/544/319), with a termination theory now learned on seven of the ten folds. That figure falls inside the 0.735–0.887 span the published systems occupy, but it must never be read as the headline: its vocabulary was chosen *after* the first run's test-side failure mode was known, on the same folds. Everything else about it is pre-registered; the vocabulary is not. Against published estimates selected as the best of several tried, the smaller fully pre-registered number is the stronger scientific claim, which is why 0.733 leads here and 0.7782 appears only with this label attached.

A negative result. Substituting the learned `close_nn` detector for the precomputed proximity predicate inside the Event-Calculus initiation search does not survive cross-validation: precision 0.1250, recall 0.0005, F1 0.0011 (1/7/1832) over the same ten folds. Only one fold commits a clause—the same `both_active & close_nn` shape the geometric predicate selects there, and probed directly against held-out ground-truth proximity that clause scores precision 1.0 at recall 0.27, so the network has learned the geometric relation. The failure is one of scale in the selection rule, not of perception: the initiation search scores coverage against transition *events*, a handful per fold, and a probability-thresholded gate newly covers fewer of them than a deterministic predicate does, so nine of ten folds reject the candidate for insufficient new coverage. We report this as a result rather than omitting it; the direct protocol above is where the learned detector currently pays off.

The clean protocol abstains. Re-run on the deduplicated, scene-family-grouped corpus, the `meeting` fluent yields no theory at all—both the Event-Calculus route and the direct route abstain on every fold, F1 0.0. The reason is visible in the data and is not a defect of the

Run	P	R	F1
Windowed folds, direct target (mean of 3 folds)			
precomputed `close`	—	—	0.7905
learned `close_nn`	—	—	**0.8276**
Distributed corpus, 10-fold CV, EC + inertia			
pre-registered	0.6580	0.8271	**0.7329**
+ termination vocab. (2nd iter.)	0.7357	0.8260	0.7782
learned `close_nn`	0.1250	0.0005	0.0011
Clean protocol (scene-family folds)			
`meeting`, both routes	—	—	abstains
`moving`, EC route	—	—	abstains
`moving`, direct route	0.5334	0.3817	0.4450

Table 2: Runs reported in this section. Micro-averages over pooled per-fold counts, except the windowed rows, which are fold means. "Abstains" is an empty theory returned by the permutation-null fit gate, not a zero score. Published figures for the same benchmark are tabulated separately in Section 9.

search. One scene family carries 1,323 of the 1,812 `meeting`-positive frames and, under honest grouping, sits in exactly one fold; the clause that explains those frames holds *only* there, covering zero of the 489 positives on all eight remaining `meeting`-bearing segments. Training with that family learns a clause that transfers nowhere; training without it cannot learn the clause at all. With eleven observable `meeting` initiations and five `moving` ones in the clean corpus, there is not enough evidence for a cross-validated theory, and the fit gate says so. That is the gate working as designed—a rule learner declining to manufacture a theory—and it is a more interesting property of the system than any F1 it reports elsewhere. The machinery is not broken under this protocol: on `moving` the direct route still selects the canonical published rule shape on nine of ten folds (micro F1 0.4450), and the `meeting` clause, applied directly to the held-out scene fold rather than transferred to it, scores frame F1 0.890.

7.5 What This Establishes, and What It Does Not

It establishes that a perceptual predicate can be learned end-to-end through symbolic credit alone and stand in for hand-set geometry without loss on a real benchmark (mean F1 0.8276 against 0.7905); that Event-Calculus rules can be induced under fully pre-registered gates and land within a couple of points of the range published dedicated Event-Calculus learners report on the corpus they share; that the benchmark corpus those numbers rest on carries a measurable duplication defect, established by a re-runnable audit; and that the fit gate abstains rather than fabricates when a leak-free split leaves the evidence too thin.

It does not establish a ranking against OLED, WOLED-ASP or hand-crafted rules, and none is claimed: the protocol deltas above are real, unquantified, and cut both ways, and the clean protocol has too few events for a cross-validated comparison to mean anything in either direction. It does not show that termination programs are learned in the pre-registered run—they are not; the empty termination theory is the dominant source of false positives. It does not show that the learned detector is a drop-in replacement for the geometric predicate everywhere; under Event-Calculus cross-validation it is not. And it is one benchmark in one domain. What it does show is that the structure search and the perception underneath it can be one differentiable object rather than two systems in sequence.

8 Epistemic and Other Reasoning Modes

The substrate generalizes beyond what is *true* or *probable*. This section covers three further modes that reuse the same CUDA join, circuit, and CDCL machinery—epistemic reasoning over world views, probabilistic aggregates, and well-founded semantics—broadening what a neural network can be integrated with while keeping data-plane execution GPU-backed.

8.1 Modal Surface and the Epistemic IR

Epistemic logic programming asks what is *known* or *possible* across the multiple stable models a program admits—the natural setting for introspection and reasoning under incomplete knowledge. A program may use four modal literals over a predicate—`know`, `possible`, `not know`, `not possible`. Rather than rewrite them away at parse time, the compiler preserves them in an Epistemic IR (EIR) between the AST and execution planning. The high-level dispatcher classifies modal dependencies before selecting Generate–Propagate–Test, a mode-specific positive-cycle fixpoint, or GPU-backed WFS. The accepted surface spans finite nested modal chains, epistemic integrity constraints (which prune candidate world views on the GPU), rules that couple several epistemic predicates, same-name multi-arity modal predicates, and recursive epistemic execution.

8.2 World-View Semantics

A world view is a non-empty set of stable models. `know p` holds when `p` appears in every model of the world view; `possible p` holds when it appears in at least one. Two semantics are selectable. Under the default FAEEL (Founded Autoepistemic Equilibrium Logic), a circular derivation contributes a tuple only when independent founded support exists. A program containing only `p :-`

`possible p.` therefore executes to an empty founded extension rather than failing with an unsupported-construct diagnostic. A Gelfond-1991-style compatibility mode admits that self-supporting tuple. Non-epistemic programs evaluate identically under both.

8.3 Epistemic Execution Routes

After EIR construction, acyclic modal programs follow a Generate–Propagate–Test schedule whose phases run as CUDA kernels: *generate* enumerates bounded candidate assumptions as device bitsets; *propagate* prunes candidates that contradict known or rejected facts; and *test* validates survivors against the program's world views, checking stable-model tuple membership on-device per arity. A positive FAEEL dependency cycle reduces to ordinary rules and reaches the founded least fixpoint in the existing recursive engine. A supported Gelfond-1991 cycle has a different plan: the selected positive `possible` gates must use the same terms as their rule heads and remain within one recursive dependency component. The runtime first relaxes those gates to compute an upper bound, then reevaluates from the extensional inputs while each gate reads a frozen snapshot from the preceding pass. GPU set comparison over all intensional relations detects the descending greatest compatibility fixpoint. Consequently, predicate-level recursion cannot preserve a tuple unless the recursive rules return that concrete tuple; candidate relations with disjoint tuple domains descend to the empty extension. Supported cycles through negation use the GPU-backed WFS alternating fixpoint. An unrelated WFS component can execute inside a Gelfond-1991 pass, but recursive negation or aggregation within the compatibility component is rejected because either would make the descending operator nonmonotone. The reduced ordinary work reuses the optimizer, WCOJ planner, and helper-splitting machinery of Section 4, and *epistemic splitting* decomposes eligible acyclic programs into independent components solved separately and recombined. Candidate validation on the Generate–Propagate–Test route reuses the on-GPU CDCL solver already built for compilation verification (Section 5)—the same GPU substrate handles satisfiability, bounded MaxSAT, and assumption-based queries—and accepted world-view evidence feeds the existing GPU-native exact path. The accepted data-plane hot paths perform no CPU fallback; transfer budgets track data movement. The host sequences Gelfond-1991 refinement and WFS iterations, while relation evaluation, snapshot copies, and convergence comparisons remain GPU-backed.

8.4 Probabilistic Aggregates

Finite aggregate queries (`count`, `sum`, `min`, `max`, `logsumexp`) are supported in both exact provenance/PIR inference and Monte Carlo execution, subject to a typed exact-domain cap that rejects domains too large for tractable enumeration. For small finite `count` domains, *aggregate lifting* replaces naive world enumeration with exact cardinality dynamic programming over a compact state set—collapsing, for example, a 2^{17}-outcome enumeration into a few hundred lifted states with identical results—and accepted `count` aggregates dispatch a GPU-native exact evaluator rather than a CPU finite-world shortcut, keeping the path device-resident.

8.5 Well-Founded Semantics

Programs that violate stratification—such as the classic `p :- not q. q :- not p.`—do not have a unique two-valued least-fixpoint interpretation. For supported shapes, xlog instead assigns three-valued truth (true, false, undefined) through Well-Founded Semantics, computing an alternating fixed point: mark the greatest unfounded set false, derive consequences true, and repeat until stable. For probabilistic programs, true atoms receive normal probability and gradient while false and undefined atoms are assigned probability zero, matching the conservative treatment of non-stratified programs. Probabilistic provenance analysis and epistemic dependency dispatch select the GPU-backed WFS route for the nonmonotone components they support; other cyclic shapes return a typed diagnostic.

9 Evaluation

This section reports measurements for xlog's deterministic, probabilistic, and neural-symbolic subsystems. Most quantitative claims here are tied to a checked-in benchmark artifact; where a figure is not, the text says so at the point the figure is reported. The subsystem measurements in Sections 9.2, 9.3 and 9.7 are *single-system ablations of xlog against its own baselines* on development hardware; Section 9.4 then adds controlled head-to-head comparisons against external engines (Scallop, ProbLog2, Soufflé) collected on separate cloud GPUs, Section 9.5 tabulates the Event-Calculus rule-induction runs of Section 7 beside the figures published for the same benchmark, and Section 9.6 isolates xlog's two design costs—verification and residency—from ordinary GPU acceleration. Throughput-target envelopes for operations measured only in isolation are maintained as an internal regression contract and are not restated here, and Section 9.9 names the capabilities that ship with correctness evidence but no timing artifact.

9.1 Methodology

All measurements were collected on an NVIDIA RTX PRO 3000 (Blackwell, 12 GB VRAM, compute capability 12.0). Rust crates are built in release mode; CUDA

modules are staged cubin-first with PTX fallback and explicit JIT warm-up; Python components use `pyxlog` with CUDA-enabled PyTorch. The benchmark harness uses Criterion.rs with the settings in Table 3, and GPU measurements include warm-up to amortize kernel-module loading and CUDA context setup.

Table 3: Benchmark harness settings.

Setting	Value
Sample size	10–100 runs (benchmark-dependent)
Warm-up	3 iterations (PTX JIT, memory-pool init)
Significance level	0.10
Noise threshold	0.05 (ignore <5% variance)
Seeding	Deterministic LCG

9.2 Worst-Case Optimal Joins

The WCOJ subsystem (Section 4) is exercised against the binary-join baseline by a paper-scale harness (`crates/xlog-integration/benches/wcoj_paper_class.rs`) over four fixtures—call-graph edges, Andersen points-to [24], `ddisasm`-style disassembly [25], and a recursive neural-symbolic mining analog—each with 4.19M input rows. The scale is deliberate: WCOJ speedups depend strongly on input size and skew, and synthetic micro-benchmarks systematically overstate them, so multi-rule recursive workloads at millions of tuples are the honest setting. Both paths must produce identical row sets before any timing is taken, and each measurement is a median over 10 runs with coefficient of variation ≤ 0.05.

Figure 4 reports the speedups: a 27.96× geometric mean, with every fixture between 26.6× and 29.6×. The run that produced these figures is not among the committed artifacts. This is an ablation against xlog's *own* binary-join baseline; Section 9.4 reports the external counterpart, a fused-counting comparison against Soufflé.

Figure 4: WCOJ vs. binary-join speedup on four paper-scale fixtures (4.19M input rows each). Single-system ablation; the dashed line is the geometric mean.

On uniform or empty inputs the adaptive classifier falls back to the binary-join chain. Peak observed allocation was about 2.3 GB, within the device's 12 GB budget.

9.3 Circuit Caching and Training

Neural-symbolic training is measured on MNIST single-digit addition (512 training images, 5 epochs, batch size 64). Circuit caching (Section 5) amortizes D4 compilation across epochs: a cache ablation over 3 seeds and 3 epochs reduced mean training time from 242.9 s (uncached) to 88.9 s (cached), a 2.74× speedup (95% CI $[2.29, 3.18]$). Table 4 summarizes the full-run timing: the first epoch pays cold-start compilation and verification, while warm epochs run only the cached forward/backward kernels. These timings were taken on the current pipeline; the run that produced them is not among the committed artifacts, which record an earlier generation of the code. The ablation is a separate experiment from this full run—its uncached arm recompiles the circuit every epoch while its cached arm compiles once, so it isolates compilation amortization; the apparent speedup therefore grows with epoch count toward the cold/warm ratio rather than being a fixed property. This 512-image protocol isolates cold/warm compilation timing and is deliberately under-trained (near-chance accuracy); held-out accuracy is measured at scale in the Scallop head-to-head (Section 9.4), where the same pipeline reaches ≈95%.

Table 4: MNIST-addition training summary. The timing rows come from the 512-image, 5-epoch training run; the circuit-cache speedup comes from a separate ablation (512 images, 3 epochs, 3 seeds) and is the mean of its three per-seed ratios, not the ratio of its means.

Metric	Value	Notes
First epoch (cold)	71.7 s	D4 compile + CDCL verify
Steady-state epoch (warm)	0.25 s	cached forward/backward path
Total training (5 epochs)	72.6 s	
Per-query cost (steady state)	0.97 ms	forward + backward; steady epoch over 256 pairs, cold epoch excluded
Circuit-cache speedup	2.74×	95% CI $[2.29, 3.18]$

9.4 Head-to-Head Comparisons

The measurements above compare xlog against its own baselines. We additionally ran controlled head-to-head comparisons against external engines—one per reasoning mode—on ephemeral cloud GPUs (RunPod RTX 3090/4090), with matched programs, data, and metrics; the artifacts are under `artifacts/head-to-head/`. The picture is deliberately honest: xlog wins clearly where its GPU-resident, bounded-memory design is the point, and does not where a mature CPU compiler is already

fast enough on small inputs. Table 5 summarizes; the paragraphs below give the detail.

Neural-symbolic: MNIST addition vs. Scallop. With identical MNISTNet, data, metric, and seeds, and held-out addition accuracy on the 10k-image test set, at 20 000 training images over 5 epochs (3 seeds) xlog and Scallop reach comparable accuracy (0.955 ± 0.003 vs. 0.950 ± 0.006), while xlog runs a steady epoch in 8.71 ± 0.09 s against Scallop's 24.35 ± 0.51 s—a 2.80× per-epoch speedup—and is faster end-to-end (108.5 s vs. 122.7 s over 5 epochs) once the one-time D4 compilation amortizes. The advantage is time and residency, not accuracy.

Probabilistic: exact inference vs. ProbLog2. On five programs (a conditioned Bayesian fragment and probabilistic reach chains) xlog's exact query probabilities match the analytic answers to zero error, identical to ProbLog2 (correctness gate $|\Delta| < 10^{-4}$). On timing, however, xlog is *not* competitive on these small programs: its per-query time is flat at ≈0.25 s (fixed GPU launch plus per-call D4 compile and CDCL verify) while ProbLog2's mature CPU compiler runs each in 0.014–0.028 s (9–19× faster). We therefore make no exact-inference *speed* claim; the probabilistic contribution is correctness-equivalent inference that is machine-verified on-device and GPU-resident, which amortizes only at scale or under repeated evaluation.

Deterministic: WCOJ triangle counting vs. Soufflé. On hub-skewed graphs, xlog's fused worst-case-optimal triangle counting produces the same triangle totals as Soufflé at every size (5.4M, 13.1M, 23.1M triangles) while running 12.6×, 23.9×, and 42.5× faster on wall-clock (0.24–0.33 s vs. 3.0–13.6 s), the gap widening with graph size. Crucially, fused counting never materializes the triangles: peak device memory stays flat at 4–15 MB across all sizes, whereas materializing the join intermediate needs 3–7 GB and exhausts the budget (OOM) at 23M triangles. This is the worst-case-optimal thesis demonstrated externally—the win is bounded intermediate memory on skewed, large-intermediate workloads, not a universal Datalog speedup (on moderate skew the advantage over xlog's own binary join is only 1.1–1.5×). The three-way artifact is marked not-fully-acceptable precisely because the enumerate-then-count arm OOMs at the largest size, which is the intended demonstration of the memory bound; the fused-vs-Soufflé correctness gate passes at every size.

9.5 Event-Calculus Rule Induction

Section 7 states the induction protocol and analyses its runs, and Table 2 there lists them; this subsection adds the two things an evaluation owes. The first is the set of figures other authors have published for the same benchmark, tabulated beside our pre-registered row. The second is the runs that do not favour the system, which are reported here for the same reason the favourable one is.

Published figures for the same benchmark. Table 6 places our pre-registered ten-fold row beside the four published frame-level `holdsAt` F1 figures for `meeting` on the whole distributed CAVIAR corpus. Ours is 0.7329 (precision 0.6580, recall 0.8271; 1516 true positives, 788 false positives, 317 false negatives), micro-averaged over pooled per-fold counts as the published evaluations are. The published figures span 0.735 to 0.887. Our figure is *below the floor of that span*, by 0.0021: a fully pre-registered run lands just under the lowest published figure, which is the hand-crafted rule set. We do not describe it as falling inside a parity band, because it does not fall inside one. The second-iteration run of Section 7 reaches 0.7782, which is inside the span, but its Event-Calculus vocabulary was chosen after the first run's test-side failure mode was known on the same folds; it is reported only with that label attached and is not the row in the table.

One asymmetry runs in the published rows' favour and we state it without drawing a conclusion from it: our gates, seeds and folds were fixed before the run that produced our row and that run was executed once, whereas the published settings are described by their authors as the best among several parameter settings they tried. We make no adjustment to any number on that basis, and we do not order the rows.

A learned detector against precomputed geometry. The result that depends on no published figure is the windowed-fold comparison with a direct per-timestep target. The search returns the same two-clause theory on every fold whether the proximity predicate is the precomputed geometric one or the learned `close_nn`. Held-out F1 per fold is 0.9215/0.6804/0.7695 with the precomputed predicate (mean 0.7905) and 0.9215/0.7918/0.7695 with the learned one (mean 0.8276): the two modes agree to the digit on folds 1 and 3, and differ only on fold 2. A perceptual predicate that was never shown a distance label therefore stands in for hand-set geometry on this protocol without costing accuracy, which is the claim Section 7 exists to make.

The same detector under Event-Calculus cross-validation: a negative result. Substituting `close_nn` for the precomputed predicate *inside* the Event-Calculus initiation search does not survive the ten-fold protocol: precision 0.1250, recall 0.0005, F1 0.0011 (1 true positive, 7 false positives, 1,832 false negatives) in 820.1 s of wall clock. Nine of the ten folds reject the candidate for insufficient new coverage; Section 7 gives the diagnosis, which is a scale mismatch in the selection rule rather than a failure of the network. The row is reported rather than omitted: an evaluation that carried only the protocol-matched row would be publishing a selection of its own runs.

Abstention under the clean protocol. Re-run

Table 5: Head-to-head summary: one external engine per reasoning mode, matched programs, data, and metrics, on ephemeral cloud GPUs. The correctness gate passes in all three; timing is the honest comparative picture. Artifacts in `artifacts/head-to-head/`.

Mode / task	**Baseline**	**Correctness**	**Timing (xlog vs. baseline)**
Neural: MNIST addition	Scallop	acc 0.955 vs. 0.950	steady epoch 8.71 s vs. 24.35 s (**2.80**× faster); faster end-to-end
Probabilistic: exact inference	ProbLog2	0 error (both)	per query ≈0.25 s vs. 0.014–0.028 s (9–19× *slower* on small programs)
Deterministic: triangle counting	Soufflé	counts match exactly	0.24–0.33 s vs. 3.0–13.6 s (**12.6**–**42.5**× faster), peak memory flat 4–15 MB

Table 6: Frame-level `holdsAt` F1 for the `meeting` fluent on the distributed CAVIAR corpus, micro-averaged over pooled counts. Rows are ordered alphabetically by system name and none is emphasised. They are deliberately *not* ordered by F1: the five protocol deltas printed beneath the table are real, unquantified, and do not all act in the same direction, so an ordering by score would assert a comparison this evidence does not support. Dashes are figures the cited table does not report.

System	**P**	**R**	**F1**	**Source**
Hand-crafted rules	0.644	0.855	0.735	OLED [22] Tab. 1(b), `EC_crisp`; the same value appears in WOLED [23] Tab. 2
OLED	0.678	0.953	0.792	OLED [22] Tab. 1(b), whole dataset
OLED, as re-run by other authors	—	—	0.782	WOLED [23] Tab. 2
WOLED-ASP	—	—	0.887	WOLED [23] Tab. 2
xlog, EC + inertia, pre-registered	0.6580	0.8271	0.7329	This work: 10-fold CV by video segment, all gates re-derived per fold (Section 7)

Protocol deltas, which must travel with this table. (i) The published runs cross-validate over windowed interpretations with subsampled negatives; ours folds over whole video segments. (ii) They learn full initiation *and* termination programs; our row has a two-literal initiation clause and an empty termination theory, which by itself accounts for 788 of 2,304 predicted-positive frames being false positives. (iii) Their rule language admits longer bodies—bottom clauses averaging some fifteen literals against our two-literal cap. (iv) Their settings are reported as the best among several tried; ours were pre-registered. (v) All rows compute F1 on `holdsAt` inferred through inertia, on the same corpus, with the same micro-aggregation—the one axis on which the protocols do match, and the reason the rows are placed side by side at all.

on the deduplicated, scene-family-grouped corpus, the `meeting` fluent returns no theory at all—the Event-Calculus route and the direct route both abstain on every fold (0/120/1812 and 0/106/1812 tp/fp/fn respectively), F1 0.0 on both. On `moving` the Event-Calculus route also abstains (0/0/3136), while the direct route selects a clause on nine of the ten folds—fold 6 abstains—for micro precision 0.5334, recall 0.3817, F1 0.4450 (1197/1047/1939). These zeros are empty theories returned by the permutation-null fit gate, not scored predictions: with eleven observable `meeting` initiations in the clean corpus the gate declines to commit a theory, which is the behaviour it was pre-registered to have.

Why the two corpora differ. A frame-level audit of every `meeting` transition event in the distributed corpus against the original ground-truth XML classifies 21 of 25 events as real, 3 as duplicates and 1 as a splice artifact, with one source video contributing 855 duplicated gold pair-frames. That defect is the reason the distributed-corpus rows carry a caveat—and also the reason the comparison in Table 6 is drawn there rather than on the clean corpus: the distributed dump is the regime the published figures share.

9.6 Isolating Verification and Residency Overhead

Two isolations separate xlog's two design costs from ordinary GPU acceleration; both run on a RunPod RTX 3090, with artifacts under `artifacts/head-to-head/`.

Verification cost. The cold epoch's compilation time is dominated by the on-GPU CDCL equivalence check, not the D4 compile. Splitting the cold compile via `warmup_breakdown()` on probabilistic reachability chains, CDCL verification accounts for 96–100% of the cold-compile time and grows with circuit size (251 ms at n=5 to 508 ms at n=40), while D4 compilation stays under 17 ms. The "expensive" cold epoch is thus almost entirely the price of the machine-checked correctness certificate (Section 5)—a cost the other systems in Table 7 do not pay because they do not verify.

Residency benefit. To isolate the transfer-free advantage from ordinary GPU acceleration we run the same neural–symbolic pipeline with and without a forced device→host→device round-trip at the neural–symbolic boundary—the transfer a CPU-reasoning hybrid pays every step—sweeping the number of neural→symbolic handoffs per step (the batched addition path, 2 to 512 handoffs). Each handoff costs a stable ≈40–56 μs round-trip; because the batched circuit compute grows sublinearly with the batch while the transfer grows linearly with the handoff count, the round-trip's share of a step *rises with scale*: 0.5% at 2 handoffs, 4% at 32, and ≈10% at 128—the standard batch-64 MNIST-addition step (7.2 ms of 72 ms)—holding near 9% at 512. On a single query the transfer is negligible, but at a realistic training batch it is already a ≈10% tax that xlog avoids: a real, measured contribution, additive to and distinct from the GPU-compute and circuit-caching advantages, and growing with the per-step neural→symbolic fan-out. The forced per-buffer copy is an upper bound (a hybrid could coalesce transfers), and the larger transfer-dominated regime—large-state Datalog fixpoints where whole relations would be shipped each iteration (Section 1)—needs a separate relation-residency ablation we leave to future work (Section 11).

9.7 Runtime Optimization

Two of the runtime optimizations of Section 4 target repeated-session workloads. The persistent hash-index manager retains built join indices across evaluations, keyed on relation generation, schema signature and device ordinal, under a byte-budgeted LRU policy: a session whose relations change little between calls reuses an index instead of rebuilding it, and a generation change invalidates the entry rather than serving a stale one. The profile-gated shared-memory chain scorer (`crates/xlog-cuda/kernels/ilp_exact.cu`) tiles the left relation through shared memory when scoring chain-shaped candidate bodies during exact rule induction, and is gated on a candidate-row threshold below which the baseline scorer runs unchanged.

Both are measured on repeated-session fixtures. The index manager records a 3.21× speedup on a build-heavy repeated-session semi-join (cached median 0.08 s vs. uncached 0.26 s) with zero tracked host transfers, and the chain scorer records a 5.58× speedup on a chain-hot fixture with a bounded 1.3% regression on the small-input control. The runs that produced these two figures are not among the committed artifacts. They are point medians on single fixtures; per-fixture dispersion is left to future work.

What committed tests pin is narrower than these two figures. For the index manager they pin the cache bookkeeping—that the key separates entries differing in relation generation, schema or device, that the byte budget evicts by LRU, and that invalidation removes the entry—and, on a repeated-session semi-join, that the index is built once and reused thereafter with no stale acceptance, while both the cached and uncached paths record zero tracked host-to-device and device-to-host calls. That zero-transfer property is asserted both in the Rust executor tests and end-to-end through the Python session API; it is read from the provider's own transfer counters, and the two paths are compared by output row count rather than by row set. For the chain scorer no committed test executes the shared-memory kernel at all: the exact-induction tests that check the native path against a reference implementation use fixtures below the gate threshold and therefore exercise only the baseline scorer, so neither the small-input fallback nor the threshold is pinned. Common-subexpression elimination

and adaptive re-optimization are validated for output equivalence rather than reported as headline speedups.

9.8 Capability Comparison

Table 7 positions xlog qualitatively against representative systems; each addresses a subset of the design space, and xlog's contribution is unifying them in one GPU-resident runtime. This table is capability coverage, not performance: quantitative head-to-head results against Scallop, ProbLog2, and Soufflé are in Section 9.4, while GPUlog, VFLog, Lobster, and DeepProbLog remain to be benchmarked directly.

The rule-induction row needs its evidence status stated, because the two paths behind it are not equally evidenced and the distinction matters. The trainable-rule-body path is carried onto an external benchmark in Section 7 and tabulated in Section 9.5, under pre-registered gates and cross-validation; it is no longer a demonstration-only surface. What that evidence does *not* supply is an ordering against a dedicated rule learner, for the protocol reasons given with Table 6. The differentiable-ILP path is not exercised there at all and remains validated by demonstrations rather than benchmarks (Section 6). Both are beta surfaces (Section 11), and the table's "yes" claims capability, not maturity.

9.9 Capabilities Carried by Correctness Evidence Only

Four capabilities added since the measured runs above ship with correctness evidence—tests that pin their behaviour—but with no committed timing artifact. We describe what each does and publish no speedup for any of them.

Exact log-evidence and the CNF-variable-to-fact map. An exact evaluation now returns the log partition function of the weighted formula, $\log Z$, alongside the per-query probabilities, so a program's evidence mass is read off directly instead of being reconstructed from marginals; it surfaces as `log_z_e` on the Python result. Paired with it, `prob_var_map()` reports which probabilistic fact each CNF variable stands for—an ordinary fact, one Bernoulli decision of an annotated disjunction's chain, or padding—so the per-variable gradient buffers can be attributed back to program facts rather than to opaque indices. The map is deliberately partial and says so: its length is the encoder's variable *capacity*, not a variable count, and it is refused with a typed error for Monte Carlo programs and for exact programs compiled through the GPU count-lift fast path, which never builds a CNF encoding and therefore has no map to report.

Same-head rule unions in one multiway pass. A predicate head defined by many rules used to fold its per-rule contributions into the head relation one union at a time, re-sorting the growing accumulator once per rule and going quadratic in the rule count—a real cost at corpus scale, where one head can carry thousands of rules. Contributions are now merged with a single multiway union per head per iteration. The regression test pins this at the `--stats` operation level, asserting that the union count does not scale with the same-head rule count (eight same-head rules record one union, not eight) while the derived rows stay byte-identical to the unbatched result.

An exact memoized-DP stage beyond enumeration capacity. The joint-constraint solver is exact by complete enumeration where a component fits the enumeration capacity. Chain-order path components that exceed it now take a memoized dynamic-programming CUDA stage over reached-domain bitsets, which is also exact: the passes are restricted forward passes, so every emitted total is a linearly accumulated `f32` and margins come only from exact passes, never from bounds. Components with wider frontiers refuse on the device with a typed status rather than silently degrading to an approximation; the pinned width gates eligibility, and the fuel meter reconciles against device-measured DP transitions instead of an estimate.

The joint-constraint carrier and its zero-copy binding. The carrier owns every solver buffer on the device. Python consumers receive DLPack [7] views of the exact producer allocations—ownership stays with xlog, and the shared runtime identity is kept alive so strict launch recorders keep recording the exported view rather than rejecting it—and drive the cold-path session lifecycle explicitly: allocate, register schema, bind signatures, solve label feasibility. The fuel meter lives inside the session, so exhaustion saturates across calls and a retry repeats the identical typed refusal instead of slipping further work past the budget; every refusal names the precondition it enforced and its literals.

9.10 CUDA Certification

The CUDA kernel certification suite covers all GPU kernel operations: 207 tests across 33 categories (25 infrastructure C01–C25 + 8 GPU-specific G01–G08), at 100% pass on the latest full certification, covering 22 kernel modules. The infrastructure categories cover toolchain validation, launch configuration, memory spaces, synchronization, warp-level operations, atomics, floating-point and determinism semantics, and host-device transfer accounting; the GPU-specific categories validate circuit forward/backward evaluation, weight injection, transfer efficiency, circuit caching, PTX robustness, and SAT/CDCL solving. Because the public repository does not rely on hosted GPUs, the suite runs through GPU-backed local release validation; the always-on CI covers formatting, linting, and no-GPU build checks.

Table 7: Capability comparison across neurosymbolic and GPU logic systems.

Dimension	DeepProbLog	Scallop	Lobster	GPUlog	xlog
Symbolic execution	CPU (Prolog)	CPU (Datalog)	GPU (Datalog)	GPU (HISA)	GPU (semi-naive)
Probabilistic inference	Exact (CPU)	Provenance (CPU)	Provenance (GPU)	None	Exact d-DNNF + MC (GPU)
Circuit verification	None	None	None	—	On-GPU CDCL
Neural integration	Prolog bridge	Differentiable	Differentiable (GPU)	None	Zero-copy, fused
Zero-copy ML interop	No	Partial	Partial	No	Yes
Rule structure induction	No	No	No	No	Yes (beta): trainable bodies, dILP
Epistemic reasoning	No	No	No	No	Yes (finite fragment)

10 Related Work

xlog draws on five lines of research—neurosymbolic programming, GPU-accelerated Datalog, probabilistic logic programming, GPU SAT, and differentiable ILP—and is, to our knowledge, the first to make the full symbolic spectrum GPU-resident and uniformly differentiable.

10.1 Neurosymbolic Programming

DeepProbLog [2] replaces selected probabilistic facts with neural-network outputs, enabling end-to-end gradient training of neural predicates within a probabilistic-logic framework; NeurASP [3] integrates network outputs as probability annotations on answer-set programs. Both perform symbolic inference on the CPU and shuttle gradients across the bus. Scallop [26] provides a differentiable Datalog with provenance-semiring reasoning [13] and a relational symbolic representation, and is the closest language-level analogue to xlog; its reasoning core, however, is CPU-based. Lobster [27] is the most directly comparable system: it GPU-accelerates neurosymbolic programs in the Scallop style and shares xlog's premise that the symbolic side belongs on the GPU. xlog is complementary along three axes: it covers a broad symbolic spectrum (exact knowledge compilation and epistemic reasoning, not only provenance-based differentiable Datalog); it *verifies* each compiled circuit's logical equivalence to its source formula on-device via CDCL; and it enforces a strict residency contract with byte-level transfer accounting. We make no head-to-head performance claim against Lobster (Section 11). Differentiable-SAT and logic-as-loss methods—SATNet [28], semantic loss [29], and Logic Tensor Networks [30]—instead couple neural models to constraints through relaxed or fuzzy surrogates; xlog couples through *exact* weighted model counting, trading their broader applicability for exact gradients and the on-device verifiability of Section 5.

10.2 GPU-Accelerated Datalog and Joins

GPUlog [4] ported bottom-up fixed-point computation to CUDA using HISA indexing and reported up to 45× speedups over CPU Datalog engines; VFLog [5] advanced this with a vertically-fused, column-oriented kernel design avoiding intermediate materialization, reporting up to 200× over CPU column engines; mnmgDatalog [31] extends GPU Datalog to multi-node, multi-GPU clusters. Most relevant to xlog's join path, Sun et al. [32] scale worst-case-optimal joins to GPUs. All target deterministic Datalog only: none supports probabilistic semantics, weighted model counting, or gradient computation over derived facts. xlog shares the columnar, kernel-fused execution model and additionally promotes eligible cyclic and multiway rules to a worst-case-optimal join path [10–12], but treats that engine as one component of a substrate that extends through circuit-based inference to neural-symbolic gradient propagation in a single address space.

10.3 Probabilistic Logic Programming

ProbLog2 [1] established the modern pipeline for exact probabilistic inference: ground the program, encode provenance as a propositional formula, compile to a tractable target (d-DNNF [15] or SDD [33]), and evaluate weighted model counts [19]. The compilers (D4 [16], c2d [34]) run as host-side subprocesses, and recent work makes such compilation *certified* [20]. xlog adopts the same compile-once/evaluate-many model but moves the entire pipeline—provenance, Tseitin encoding, D4 compilation, equivalence verification, and forward/backward evaluation—onto the GPU, so probabilistic inference shares one address space with the neural and deterministic layers.

10.4 GPU SAT

ParaFROST [6] parallelizes CDCL clause-database simplification on the GPU while retaining sequential conflict-driven search on the CPU; FastFourierSAT [35] reformulates SAT as continuous optimization and applies GPU local search, an incomplete solver for satisfiable instances. Both are standalone competition solvers. xlog's GPU CDCL module instead serves as a verification component inside knowledge compilation (Section 5), proving circuit–formula equivalence via two UNSAT proofs and providing the certificates the probabilistic and epistemic backends rely on.

10.5 Differentiable ILP

Differentiable ILP [36] casts first-order rule induction as differentiable optimization, scoring candidate rules by soft forward-chaining; dense materialization over the rule space scales cubically in the number of constants, and implementations remain CPU-based. xlog's dILP subsystem (Section 6) replaces dense materialization with sparse GPU bitmasks and on-device scatter–gather credit assignment, avoiding the cubic blowup while preserving differentiability.

10.6 Logic Programming Languages

xlog inherits ideas from a long line of logic languages but is the first to target GPU execution of the full surface. Prolog [37] offers unification and metaprogramming on a CPU interpreter; Mercury [38] introduced strong typing and modes, compiling to native code; Soufflé [39] is a typed Datalog compiling to C++ for program analysis; and clingo [40] provides answer-set semantics on CPU grounders and solvers. xlog adopts the typed-predicate discipline but targets device kernels, and unifies typed Datalog, probabilistic facts, neural predicates, and epistemic operators in one language compiled entirely to the GPU.

11 Limitations and Future Work

11.1 Current Limitations

NVIDIA GPU required. The entire execution stack is written in CUDA C; there is no OpenCL, Metal, or ROCm backend. This is a direct consequence of the GPU-native design: the persistent-index manager, the recorded-launch discipline, and the in-kernel hash-consing of the compilation pipeline all depend on CUDA-specific primitives (cooperative groups, warp-level operations, unified virtual addressing), and a lowest-common-denominator portability layer would erode the zero-transfer performance model that is the system's reason for existing. Users without an NVIDIA GPU cannot run xlog.

All data must fit in GPU memory. Relations, indices, circuit buffers, and Monte Carlo sample arrays are allocated entirely on-device; there is no out-of-core spilling. A working set exceeding VRAM fails with `RESOURCE_EXHAUSTED` rather than degrading silently. For most Datalog workloads this is not a bottleneck—a 24 GB GPU holds billions of 8-byte tuples—but large knowledge graphs or high-sample-count Monte Carlo inference can reach the ceiling.

Beta and bounded surfaces. The differentiable-ILP subsystem (Section 6) is beta: the search space is restricted to definite programs, convergence is sensitive to learning-rate schedules, and its API may change. The trainable-rule-body surface of Section 6.4—mixed neural/symbolic bodies, existential joins, and joint multi-rule mixtures—is at the same maturity: it is carried onto a public benchmark under pre-registered gates and cross-validation in Section 7 rather than resting on demonstrations, but its accuracy and training cost have not yet been established through head-to-head comparison on standard relational benchmarks, which we leave to future work alongside the broader neurosymbolic-breadth benchmarks. The epistemic runtime (Section 8) executes its accepted data-plane work on the GPU, but the broader solver portfolio (MaxSAT/portfolio services) and the end-to-end probabilistic–epistemic evidence path are still being broadened. The negated-modal WFS route and Gelfond-1991 compatibility refinement are GPU-backed but host-orchestrated. Positive modal dependency cycles have defined execution: FAEEL computes a founded least fixpoint, including an empty extension for an unseeded cycle, and supported Gelfond-1991 positive `possible` cycles compute the greatest compatible set of concrete tuples from an upper bound and frozen snapshots. The shared recursion-depth setting bounds both descending compatibility refinements and WFS iterations. Fail-closed boundaries remain for raw lowering of modal literals, recursive epistemic programs that also contain modal integrity constraints, WFS shapes outside the supported negated-modal plan, unbounded tuple-key forms, and recursive negation or aggregation within a Gelfond-1991 compatibility component. The Python batch-query helpers are typed but do not yet accept floating-point uploads, a convenience-API gap rather than a core execution limit.

Partial head-to-head coverage. Section 9.4 reports controlled comparisons on identical programs and data against one external engine per reasoning mode—Scallop (neural-symbolic), ProbLog2 (probabilistic), and Soufflé (deterministic)—and these establish the honest picture: comparable accuracy with a 2.8× per-epoch training speedup over Scallop, correctness-equivalent but *slower* exact inference than ProbLog2 on small programs, and 12–42× bounded-memory triangle counting over Soufflé on skewed graphs. Coverage is still partial. The most directly comparable GPU-neurosymbolic system, Lobster, and the GPU Datalog engines GPUlog and VFLog have not been built and benchmarked here; DeepProbLog has

not been run under a matched (non-timeout) protocol; and the head-to-head runs use single cloud-GPU configurations rather than a cross-hardware sweep. Broader multi-system, multi-hardware comparison remains future work.

Bounds of the Event-Calculus induction evidence. The rule-induction results of Section 7 are bounded in four ways. First, the leak-free protocol is evidence-starved. The deduplicated, scene-family-grouped corpus contains eleven observable `meeting` initiations and five `moving` ones, and on `meeting` the search abstains on every fold under both the Event-Calculus and the direct route; the permutation-null fit gate is behaving as pre-registered, but the consequence is that this protocol yields no induced `meeting` theory to evaluate and supports no conclusion about that fluent in either direction. Second, the corpus on which the protocol-matched numbers are computed—the dump distributed with the published Event-Calculus learners—carries a measured duplication defect: a frame-level audit against the original ground-truth XML classifies 21 of 25 `meeting` transition events as real, 3 as duplicates and 1 as a splice artifact, with a single source video contributing 855 duplicated gold pair-frames. We report there because it is the regime the published figures share, and every figure computed on it inherits that caveat. Third, no ranking against OLED, WOLED-ASP or the hand-crafted rule set follows from Table 6, and none is claimed: the five protocol deltas printed with that table—whole-segment folds against windowed interpretations with subsampled negatives, an empty termination theory against learned initiation *and* termination programs, a two-literal body cap against bottom clauses averaging some fifteen literals, and pre-registered settings against settings reported as the best of several tried—are real, unquantified, and do not act in the same direction, while the clean protocol has too few events to settle any of them. Learning termination programs and lifting the body-length cap, which would remove two of those deltas, is future work. Fourth, a disclosure about our own two figures: the headline 0.7329 is fully pre-registered—gates, seeds and folds fixed before the run, which was executed once—whereas the 0.7782 variant differs from it in exactly one respect, its Event-Calculus vocabulary having been chosen after the first run's failure mode was known on the same folds. That makes the higher figure one disclosed adaptive iteration rather than a pre-registered result; it is reported only with that label and is never the row we place beside a published number.

11.2 Future Work

Out-of-core execution. A streaming mode would partition relations across host and device, paging tiles through GPU memory in fixpoint-iteration order, removing the single-GPU-memory ceiling at the cost of added transfers.

Multi-GPU partitioned evaluation. Inspired by mnmgDatalog's [31] radix-hash partitioning and GPU-aware all-to-all shuffle, a multi-GPU backend would distribute relations across devices and coordinate joins over NVLink or PCIe.

Deeper epistemic residency. The remaining epistemic work is stronger residency and coverage: a device-resident, no-host-interaction well-founded path, broader non-finite tuple-key forms, and semantic forms beyond the current finite accepted surface.

References

[1] Daan Fierens et al. "Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas". In: *Theory and Practice of Logic Programming* 15.3 (2015), pp. 358–401.

[2] Robin Manhaeve et al. "DeepProbLog: Neural Probabilistic Logic Programming". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[3] Zhun Yang, Adam Ishay, and Joohyung Lee. "NeurASP: Embracing Neural Networks into Answer Set Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*. 2020.

[4] Yihao Sun et al. "Optimizing Datalog for the GPU". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2025. DOI: 10.1145/3669940.3707274.

[5] Yihao Sun et al. "Column-Oriented Datalog on the GPU". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 15177–15185. DOI: 10.1609/aaai.v39i14.33665.

[6] Muhammad Osama, Anton Wijs, and Armin Biere. "SAT Solving with GPU Accelerated Inprocessing". In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 12651. Lecture Notes in Computer Science. Springer, 2021, pp. 133–151. DOI: 10.1007/978-3-030-72016-2_8.

[7] *DLPack: Open In Memory Tensor Structure*. https://github.com/dmlc/dlpack. Accessed 2026.

[8] *cudarc: Safe Rust Wrapper around CUDA*. https://github.com/coreylowman/cudarc. Accessed 2026.

[9] *PyO3: Rust Bindings for Python*. https://pyo3.rs. Accessed 2026.

[10] Hung Q. Ngo et al. "Worst-case Optimal Join Algorithms". In: *Journal of the ACM* 65.3 (2018), 16:1–16:40.

[11] Todd L. Veldhuizen. "Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm". In: *International Conference on Database Theory (ICDT)*. 2014, pp. 96–106.

[12] Yisu Remy Wang, Max Willsey, and Dan Suciu. "Free Join: Unifying Worst-Case Optimal and Traditional Joins". In: *Proceedings of the ACM on Management of Data (SIGMOD)* 1.2 (2023), 150:1–150:23.

[13] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. "Provenance Semirings". In: *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. 2007, pp. 31–40.

[14] Grigori S. Tseitin. "On the Complexity of Derivation in Propositional Calculus". In: *Studies in Constructive Mathematics and Mathematical Logic* (1968).

[15] Adnan Darwiche. "Decomposable Negation Normal Form". In: *Journal of the ACM* 48.4 (2001), pp. 608–647.

[16] Jean-Marie Lagniez and Pierre Marquis. "An Improved Decision-DNNF Compiler". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI)*. 2017.

[17] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference (DAC)*. 2001, pp. 530–535.

[18] Adnan Darwiche and Pierre Marquis. "A Knowledge Compilation Map". In: *Journal of Artificial Intelligence Research* 17 (2002), pp. 229–264.

[19] Mark Chavira and Adnan Darwiche. "On Probabilistic Inference by Weighted Model Counting". In: *Artificial Intelligence* 172.6-7 (2008), pp. 772–799.

[20] Florent Capelli. "Knowledge Compilation Languages as Proof Systems". In: *Theory and Applications of Satisfiability Testing (SAT)*. Vol. 11628. Lecture Notes in Computer Science. Springer, 2019, pp. 90–99.

[21] *Apache Arrow: Cross-Language Development Platform for In-Memory Data*. https://arrow.apache.org. Accessed 2026.

[22] Nikos Katzouris, Alexander Artikis, and Georgios Paliouras. "Online Learning of Event Definitions". In: *Theory and Practice of Logic Programming* (2016). arXiv: 1608.00100.

[23] Nikos Katzouris and Alexander Artikis. "Online Learning Probabilistic Event Calculus Theories in Answer Set Programming". In: *Theory and Practice of Logic Programming* (2021). arXiv: 2104.00158.

[24] Lars Ole Andersen. "Program Analysis and Specialization for the C Programming Language". PhD thesis. University of Copenhagen, 1994.

[25] Antonio Flores-Montoya and Eric Schulte. "Datalog Disassembly". In: *USENIX Security Symposium*. 2020, pp. 1075–1092.

[26] Ziyang Li, Jiani Huang, and Mayur Naik. "Scallop: A Language for Neurosymbolic Programming". In: *Proceedings of the ACM on Programming Languages (PLDI)* 7 (2023), 166:1–166:25.

[27] Paul Biberstein et al. "Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming". In: *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. arXiv:2503.21937. 2026.

[28] Po-Wei Wang et al. "SATNet: Bridging Deep Learning and Logical Reasoning Using a Differentiable Satisfiability Solver". In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 6545–6554.

[29] Jingyi Xu et al. "A Semantic Loss Function for Deep Learning with Symbolic Knowledge". In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*. 2018, pp. 5502–5511.

[30] Samy Badreddine et al. "Logic Tensor Networks". In: *Artificial Intelligence* 303 (2022), p. 103649.

[31] Ahmedur Rahman Shovon et al. "Multi-Node Multi-GPU Datalog". In: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*. 2025. DOI: 10.1145/3721145.3730431.

[32] Yihao Sun et al. *Scaling Worst-Case Optimal Datalog to GPUs*. arXiv:2604.20073. 2026.

[33] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, pp. 819–826.

[34] Adnan Darwiche. "New Advances in Compiling CNF into Decomposable Negation Normal Form". In: *Proceedings of the 16th European Conference on Artificial Intelligence (ECAI)*. 2004.

[35] Yunuo Cen, Zhiwei Zhang, and Xuanyao Fong. *Massively Parallel Continuous Local Search for Hybrid SAT Solving on GPUs*. arXiv:2308.15020. 2023.

[36] Richard Evans and Edward Grefenstette. "Learning Explanatory Rules from Noisy Data". In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 1–64.

[37] Jan Wielemaker et al. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[38] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. "The Execution Algorithm of Mercury, an Efficient Purely Declarative Logic Programming Language". In: *Journal of Logic Programming* 29.1-3 (1996), pp. 17–64.

[39] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. "Soufflé: On Synthesis of Program Analyzers". In: *Computer Aided Verification (CAV)*. Springer, 2016, pp. 422–430.

[40] Martin Gebser et al. "Multi-shot ASP Solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.